TU WIEN Informatics

Kartographische Generalisierung von Stromleitungsnetzen

DIPLOMARBEIT

zur Erlangung des akademischen Grades

Diplom-Ingenieur

im Rahmen des Studiums

Software Engineering & Internet Computing

eingereicht von

Marco Fähndrich, BSc.
Matrikelnummer 01625748

an der Fakultät für Informatik

der Technischen Universität Wien

Betreuung: Univ.Prof. Dipl.-Inform. Dr.rer.nat. Martin Nöllenburg
Zweitbetreuung: Senior Lecturer Dipl.-Ing. Dr. Florian Ledermann

Wien, 22. August 2025

Marco Fähndrich	Martin Nöllenburg

TU WIEN Informatics

Generalizing Power Grid Network Maps

DIPLOMA THESIS

submitted in partial fulfillment of the requirements for the degree of

Diplom-Ingenieur

in

Software Engineering & Internet Computing

by

Marco Fähndrich, BSc.
Registration Number 01625748

to the Faculty of Informatics

at the TU Wien

Advisor: Univ.Prof. Dipl.-Inform. Dr.rer.nat. Martin Nöllenburg
Second advisor: Senior Lecturer Dipl.-Ing. Dr. Florian Ledermann

Vienna, August 22, 2025

Marco Fähndrich | Martin Nöllenburg

Technische Universität Wien
A-1040 Wien ▪ Karlsplatz 13 ▪ Tel. +43-1-58801-0 ▪ www.tuwien.at

Erklärung zur Verfassung der Arbeit

Marco Fähndrich, BSc.

Hiermit erkläre ich, dass ich diese Arbeit selbständig verfasst habe, dass ich die verwendeten Quellen und Hilfsmittel vollständig angegeben habe und dass ich die Stellen der Arbeit – einschließlich Tabellen, Karten und Abbildungen –, die anderen Werken oder dem Internet im Wortlaut oder dem Sinn nach entnommen sind, auf jeden Fall unter Angabe der Quelle als Entlehnung kenntlich gemacht habe.
Ich erkläre weiters, dass ich mich generativer KI-Tools lediglich als Hilfsmittel bedient habe und in der vorliegenden Arbeit mein gestalterischer Einfluss überwiegt. Im Anhang „Übersicht verwendeter Hilfsmittel“ habe ich alle generativen KI-Tools gelistet, die verwendet wurden, und angegeben, wo und wie sie verwendet wurden. Für Textpassagen, die ohne substantielle Änderungen übernommen wurden, haben ich jeweils die von mir formulierten Eingaben (Prompts) und die verwendete IT- Anwendung mit ihrem Produktnamen und Versionsnummer/Datum angegeben.

Wien, 22. August 2025

Marco Fähndrich

Danksagung

An dieser Stelle möchte ich allen Personen meinen Dank aussprechen, die mich während der Arbeit an dieser Diplomarbeit unterstützt haben.

Mein besonderer Dank gilt meinen Betreuern, Martin Nöllenburg und Florian Ledermann, für ihre wertvolle und kontinuierliche Unterstützung sowie für die fachliche Begleitung während des gesamten Schreibprozesses.

Ebenso danke ich allen Beteiligten der Wiener Netze GmbH, die diese Arbeit erst ermöglicht haben. Besonders möchte ich Krahofer Markus, Rockenbauer Rainer und Prager Christian danken, die zusammen mit mir die Anforderungen erarbeitet haben und ihr Wissen großzügig mit mir geteilt haben.

Mein Dank geht auch an meine Studienkollegen Dominik, Alex und Bernhard, die den Studienalltag bereichert haben, sowie an alle weiteren Mitstudierenden, die ich während meiner Studienzeit kennenlernen durfte.

Schließlich danke ich meiner Freundin Isabel für ihre Unterstützung und ihr Verständnis in stressigen Phasen, sowie meinen Eltern Claudia und Michael, sowie allen anderen Familienangehörigen, die mich während meines gesamten Studiums tatkräftig unterstützt haben und ohne deren Hilfe diese Arbeit nicht möglich gewesen wäre.

Acknowledgements

I want to express my gratitude to everyone who supported me during the completion of this thesis.

My special thanks go to my supervisors, Martin Nöllenburg and Florian Ledermann, for their valuable and continuous guidance, as well as their expert advice throughout the entire writing process.

I am also grateful to all members of Wiener Netze GmbH who made this work possible. In particular, I would like to thank Markus Krahofer, Rainer Rockenbauer, and Christian Prager for their time working on the requirements and for generously sharing their expertise with me.

I want to thank my fellow students Dominik, Alex, and Bernhard for making student life so enjoyable, as well as all other peers I had the pleasure to meet during my studies.

Finally, I am deeply grateful to my girlfriend Isabel for her support and patience during stressful times, and to my parents Claudia and Michael for their unwavering support throughout my studies, without which this thesis would not have been possible.

Kurzfassung

Stromnetzpläne werden üblicherweise in geografischen Informationssystemen (GIS) als Vektordaten modelliert, wobei einzelne Leitungen mit einer Genauigkeit im Bereich weniger Zentimeter erfasst werden. Eine Herausforderung ergibt sich bei der Darstellung solcher Pläne in kleineren Maßstäben: Die Leitungen überlagern einander, was zu einer unleserlichen Kartendarstellung führt.

Diese Arbeit präsentiert die Entwicklung eines Generalisierungsalgorithmus für Stromleitungspläne, der die Eingangsdaten in eine vereinfachte Darstellung überführt, um die Lesbarkeit bei kleineren Maßstäben zu verbessern. Die Wiener Netze stellten dafür reale Netzdaten zur Verfügung und fungierten im Rahmen eines Design-Science-Research-Prozesses (DSR) als Stakeholder bei der Anforderungserhebung.

Der Algorithmus gliedert sich in drei Phasen: (i) Vorverarbeitung, bei der die Eingangsdaten eingelesen und aufbereitet werden, (ii) Konstruktion eines Trassennetzwerks in Form eines Graphen, (iii) Erzeugung der finalen Darstellung durch versetztes Zeichnen erkannter Parallelen auf Basis eines konfigurierbaren Maßstabsparameters.

Das Ergebnis wurde anhand von drei Metriken evaluiert: Fréchet-Distanz, Anzahl akuter Winkel und Anzahl von Kreuzungen. Während der Evaluation wurden mehrere Problembereiche identifiziert: 21,4% der Leitungen konnten nicht erfolgreich verarbeitet werden, da bereits in der Konstruktionsphase Verbindungen fehlten. Zudem wurde deutlich, dass für sehr kleine Maßstäbe eine zusätzliche Verdrängungsstrategie notwendig ist. Die Performanz des Algorithmus erwies sich als zufriedenstellend: Der gesamte Datensatz konnte in 1 Stunde und 56 Minuten verarbeitet werden.

Abschließend wurden die definierten Anforderungen überprüft: Von insgesamt zwölf Anforderungen wurden vier vollständig erfüllt, vier teilweise erfüllt und drei nicht erfüllt. Eine Anforderung konnte aufgrund unzureichender Spezifikation nicht evaluiert werden.

Abstract

Power grid maps are commonly modeled in geographic information systems (GIS) as vector data, where single power lines are drawn geographically accurately with distances of a few centimeters. A challenge arises when these detailed plans need to be viewed at smaller map scales, as the power lines are prone to overlaying and cluttering, which results in an unreadable map representation.

This thesis presents the development of a power line generalization algorithm, which transforms the original plan into a generalized map, readable at smaller output scales. Wiener Netze provided real-world network data of the Viennese power grid, and also guided the development by establishing requirements during the Design Science Research (DSR) process.

Der Algorithmus gliedert sich in drei Phasen: (i) preprocessing, where the input data is read and prepared (ii) constructing a line graph from the preprocessed data (iii) generating the final map by offsetting detected parallel lines according to a configurable scale parameter.

The algorithm was then evaluated by three metrics: Fréchet distance, acute angles count, and crossing counts. During evaluation, several problem areas were identified. 21.4% of the total lines could not be processed successfully due to missing connections during the line graph construction. For support of smaller scales, the need for a simplification strategy has been highlighted. In terms of performance, the algorithm processes the complete dataset in 1 hour and 56 minutes.

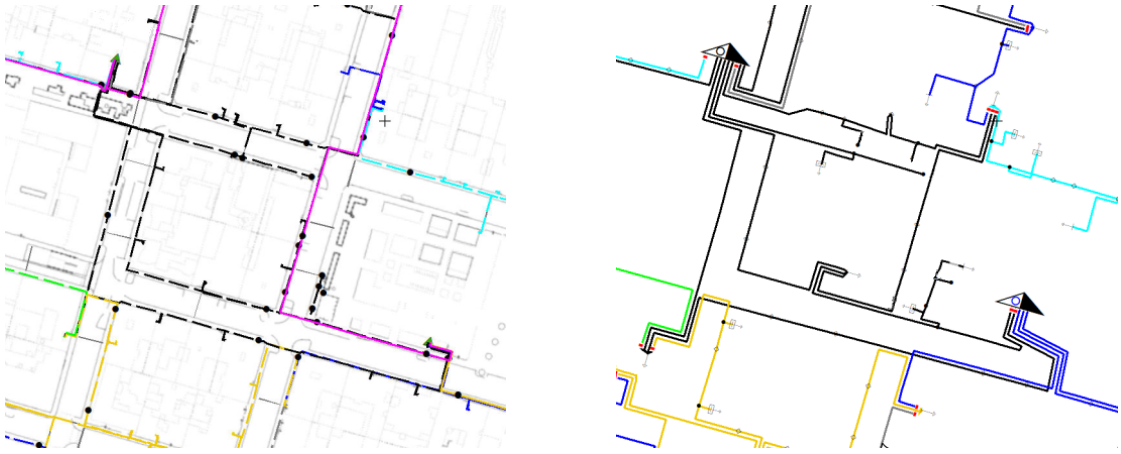
Finally, the established requirements were evaluated: out of twelve in total, four were fully met, four were partially met, and three were not met. One requirement could not be evaluated due to insufficient specification.

Contents

CHAPTER 1

Introduction

Power grids serve as critical infrastructure in urban environments, facilitating the transmission and distribution of electricity to meet the demands of modern society. The maintenance and management of these networks are entrusted to network operators, who rely on accurate plans of the network topology to ensure efficient operation and timely maintenance. Such plans are often modeled in geographic information systems (GIS), as vector data, where single power lines are drawn geographically accurately with distances of a few centimeters.



(a) Ungeneralized (b) Generalized

Figure 1.1: Drawing of power line plan

A challenge arises when these detailed plans need to be viewed at smaller map scales, as the power lines are prone to overlaying and cluttering, which results in an unreadable map representation, as seen in Figure 1.1a. To overcome the issue, cartographic generalization is used to produce a readable drawing as seen in Figure 1.1b. “Map generalization is the

name of the process that simplifies the representation of geographical data to produce a map at a certain scale with a defined and readable legend.", as defined by Ruas [32]. During the process, the original data is abstracted, and objects are simplified or removed to show an appropriate amount of information. The smaller the scale of the map, the more details have to be manipulated to fit on the available space [32].

When visualizing networks whose elements have geographic locations, the disciplines of cartography and graph drawing converge [40]. The latter, graph drawing, is the field in computer science and mathematics, with the aim to produce two-dimensional visualizations of graphs, so-called graph drawings [17]. In the context of the provided example, we can represent the given network as a node-link diagram. The **links** correspond to the cables, or more specifically, each link is a cable segment. The **nodes** represent intersection points in the network, such as street junctions, substations, and house connections. Substations transform the voltage to be consumed by the residences, which are linked to the power grid through house connections, thus both elements are endpoints in the network. The power lines are modeled as routes running from one endpoint to another, defining a route as the collection of multiple cable segments.

With the given setup, the generation of the generalized drawing can be formulated as a graph drawing problem. A graph with nodes and edges describes the topology of a network, but has no defined layout. Thus, the drawing of a graph is the embedding of nodes and edges in a plane. However, in our case, we are already given a geographically embedded input through GIS data. The initial layout reflects the real-world power grid and must not be arbitrarily distorted. Therefore, while the problem can be framed as a graph drawing problem, we have to enforce strong constraints to preserve the overall structure of the original drawing. The goal is then to optimize the quality and readability for a given scale, without losing geographic validity. The desired properties are called quality criteria [22] and depend on the use case of the drawing. Therefore, by carefully defining quality criteria and adhering to drawing conventions in collaboration with stakeholders, the process can yield graph drawings that effectively serve their intended purpose.

1.1 Research Objectives

This thesis addresses the problem of generalizing power grid network data at a metropolitan level by designing, implementing, and evaluating a suitable algorithm.

The research objectives of this thesis are:

- The underlying problem is identified and formally modeled.
- An efficient algorithm is designed and implemented to solve the generalization task.
- A framework for evaluating the results is developed, with defined methods and quality criteria.

The work is approached as an interdisciplinary collaboration between informatics and cartography, where the latter is represented by Florian Ledermann from the Research Unit of Cartography at TU Wien. Real network data will be provided by Wiener Netze, a network operator maintaining the Viennese power network.

1.2 Methodology

The methodology employed in this work adheres to the principles of Design Science Research (DSR), as defined by Brocke, Hevner, and Maedche [13]. DSR is a problem-solving paradigm whose main goal is to gain knowledge through the development of an artifact and its application in a real-world scenario. The Methodology and process are introduced in Chapter 3.

1.3 Contribution to the State of the Art

While there exist commercial solutions for generalizing power grid network data, there is a lack of scientific approaches on this exact topic. However, similar challenges have been explored in related domains. For example, Sadahiro, Tanabe, Pierre, and Fujii [33] generated bus route maps from bus stop data. Bast, Brosi, and Storandt [7] developed a framework for drawing metro maps based on metro network data. Similarly, Mizutani, Kim, Yamamoto, and Takahashi [27] proposed a method for generating schematic bus route network maps. Related work is further discussed in Section 2.2.

The contribution to the state of the art is a method for generating generalized power grid network data that integrates principles from existing approaches while addressing the unique challenges specific to power grid networks. Unlike previous methods, which primarily focus on transportation systems, this thesis extends the applicability of such techniques to the domain of electrical infrastructure. Furthermore, the requirement to apply the algorithm at multiple scales introduces a distinctive challenge that is not addressed in the referenced works.

1.4 Context within the Master Program

The thesis fits best into the algorithmics module of the Software Engineering & Internet Computing Master's program, with the Algorithmic Geometry (192.133) course providing foundational knowledge. Notably, the Graph Drawing Algorithms (192.141) course was particularly valuable, both in terms of essential knowledge acquisition and as a primary source of inspiration for the current direction of my work.

As for its relevance to other Master's programs at TU Wien, Visual Computing may be the best fit, due to its focus on visualizing data and algorithms.

1.5 Data Confidentiality

This thesis uses real network data from the Viennese power grid, provided by Wiener Netze. As the network plans represent critical infrastructure, there is a need to balance openness for research purposes with confidentiality requirements.

To address this, all result visualizations are presented at limited scales and do not include any GPS coordinates or identifiable geographic locations. This approach ensures that sensitive information remains protected while still allowing meaningful insights into the algorithm's performance.

1.6 Outline of the Thesis

This thesis is structured as follows:

- Chapter 2 - Related Work: Introduction of underlying concepts and state of the art research in related areas.
- Chapter 3 - Requirements: Defines the functional and technical requirements for the algorithm and describes the requirements process, as well as the underlying methodology.
- Chapter 4 - Algorithm: Describes the design and implementation of the algorithm for power grid network generalization.
- Chapter 5 - Evaluation: Defines the evaluation framework and presents insights and results.
- Chapter 6 - Conclusion: Summarizes the contributions of the thesis, discusses limitations, evaluates the extent to which the defined requirements have been fulfilled, and outlines directions for future research.

CHAPTER 2

Related Work

This chapter introduces the underlying foundations for understanding the principles and techniques discussed in this work. First, we give an overview of relevant techniques and concepts, and then examine related research in the field.

2.1 Background

This work relies on geometric operations and graph-theoretic concepts, starting with the explanation of the *Fréchet distance* and followed by the concept of *Free Space*. Both are crucial for the field of *map construction*, which is the domain focused on constructing networks from individual paths, often GPS-trajectories. [2]

2.1.1 Map Generalization

Cartographic generalization, or map generalization, is the process of transforming a map at a given input scale to produce a map that remains readable at a smaller output scale. Generalization operators are the atomic operations used in cartographic generalization. There is no commonly accepted standard set of generalization operators [34]. The following are listed by Schiewe [34]:

- Simplification - reduces the geometric complexity of features by decreasing the number of points.
- Elimination / Selection - elimination removes objects from the output; selection preserves them.
- Scaling - changes the size of an element, often to meet minimum dimensions or improve visibility.

- Emphasis - highlights specific elements, e.g., by using thicker lines or distinct colors.
- Displacement - shifts objects when their relative positions no longer reflect real-world relationships.
- Aggregation - merges related objects based on geometric, topological, or thematic criteria.
- Collapse - reduces dimensionality, e.g., converting an area (polygon) to a line or point.

The greater the difference between the input and output scales, the more details must be compressed into the new representation, which increases the complexity of the generalization process. As shown in Figure 2.1, some information is often lost to emphasize important details. In this example, some houses are aggregated to represent a block of buildings, but the actual number of buildings is lost [32].

In a map, each feature is typically represented by a distinct, recognizable symbol, such as a square for a house or a line for a street. During the process of generalization, this symbol must remain readable; if the square becomes as small as a point, it will no longer be identifiable as such. Such graphical constraints have to be respected during the generalization process and can vary depending on the represented elements or the context of the map. [32]

The term map generalization refers to the manual process, while automatic map generalization describes the same process, which is carried out by a computer program instead of a human. One of the first approaches to fully automated map generalization was around 2012 [36]. Furthermore, deep learning has generated new research interests in map generalization, as discussed in [37].

Generalization is typically divided into two categories: model generalization and cartographic generalization. While cartographic generalization focuses on improving the readability and visual clarity of maps, model generalization targets the structural properties of the underlying geographic model [29]. Although our algorithm operates on the model level, it is best classified as map generalization, since the main objective is to enhance the readability of the output map.

2.1.2 Geometric Foundations

This work uses basic geometric concepts as the foundation for modeling power grid elements:

- **Point** - A zero-dimensional geometric object defined by coordinates (x, y) in a two-dimensional space.
- **Polyline** - A sequence of points, which defines a connected path in two-dimensional space.

(a) scale 1:25K (b) scale 1:50K

Figure 2.1: Map before and after generalization (from [32])

2.1.3 Graph-Theoretic Concepts

While basic geometric objects are used to represent elements of the network, the work relies on Graph theory to formally define the structural properties of the power grid. The following definitions are used throughout this thesis:

- **Graph** - A graph $G = (V, E)$ consists of a set of vertices V and a set of edges E connecting pairs of vertices. Graphs can be directed or undirected, depending on whether the edges have an orientation.
- **Line Graph** - A graph derived from an input graph where each edge is represented as a vertex, and two vertices are connected if their corresponding edges share a common endpoint. In this work, parallel edges in the input graph are combined into a single edge in the line graph.
- **Geometric graph** - A graph, which has a fixed embedding in space(2D in this work). Vertices correspond to points in a two-dimensional space. An edge represents a line connecting both of its endpoints.

Information on the term *line graph*

The term *line graph* refers to a general concept in graph theory where edges from the source graph are replaced with vertices and vice versa. In this work, we refer to the line graph as defined in Subsection 2.1.3; the term was adopted from [7].

2.1.4 Spatial Data & Queries

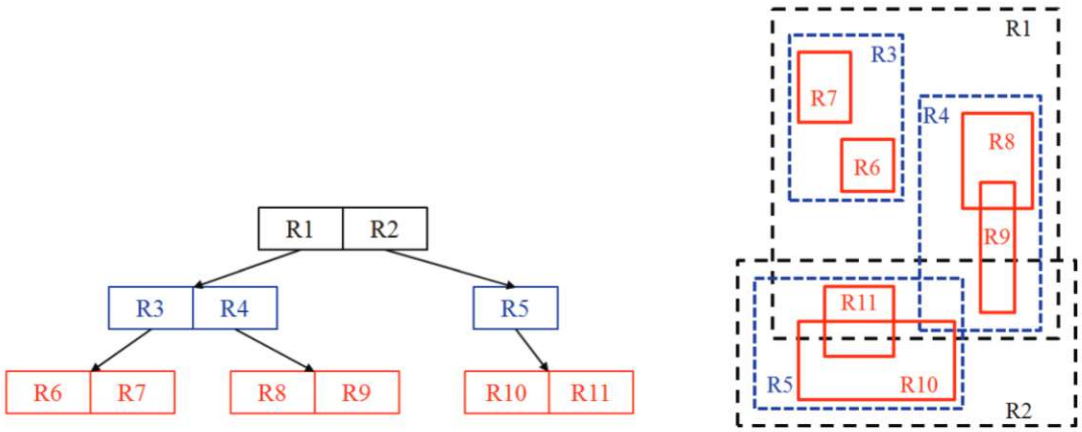
Efficient data structures and methods capable of handling large amounts of spatial data are essential for developing geometric algorithms. These will be described in the following chapter.

R*-tree

The R*-tree [10] is an advanced variant of the R-tree [23], specifically designed to improve performance in indexing and querying spatial objects. It is a hierarchical data structure that efficiently stores and manages spatial objects, such as points, line segments, and rectangles [10].

The R-tree stores nearby objects in subtrees, where each node in the tree has a bounding box that encloses all its child elements, as shown in Figure 2.2. During a search, the algorithm checks the bounding boxes of parent nodes to determine whether it should continue exploring a specific subtree. If a bounding box does not intersect the search query, the corresponding subtree can be safely pruned, making the search more efficient. Because of its dynamic nature, it allows insertions and deletions can be intermixed with queries at any time. [10]

Compared with the original R-tree [23], the R*-tree [10] reduces the area, perimeter, and overlap of bounding rectangles, and it uses forced reinsertion to keep the data well distributed. These changes boost query performance but demand slightly more storage. Both variants share the same search algorithm: a balanced tree returns a height-n node in $O(\log n)$ time, while a worst-case degenerate tree degrades to $O(n)$. Although the R*-tree and the classic R-tree have the same asymptotic insertion cost of $O(\log n)$, the R*-tree's improved insertion strategy delivers consistently faster queries in practice [10].



(a) Hierarchical view (b) Bounding box view

Figure 2.2: Visualization of r-tree (from [14])

DBScan

DBScan is a density-based clustering algorithm proposed by Ester, Kriegel, Sander, and Xu [21]. Although initially introduced in 1996, it is still used widely nowadays and in different fields such as machine learning and data mining, but also for spatial data. [35] It was the first density-based clustering algorithm designed for spatial and non-spatial high-dimensional databases. It clusters data of arbitrary shapes while handling noise effectively. The core idea is that a point belongs to a cluster if its neighborhood, within a radius (ϵ), contains at least a minimum number of points (MinPts). This ensures that clusters are formed based on dense regions in the dataset. [24]

The model identifies three categories of points based on their spatial relationships: core points, border points, and noise. Core points are those that have at least *MinPts* neighbors within a distance of *Eps*; we call them density reachable. Border points are not core points themselves but fall within the *Eps* range of at least one core point. Finally, noise points are those that do not fall within the *Eps* range of any other point.

As illustrated in Figure 2.3, core points are shown in red, border points in yellow, and noise points in blue. The *MinPts* value is set to 4 in the example.

The algorithm iterates through all points in the dataset. For each point, a range query is executed to identify all reachable neighbors within a specified distance. These neighbors are then used to build a cluster, which is expanded further by recursively adding neighbors of the current set of points until no new neighbors are found [35]. Efficient execution of range queries can be achieved using index structures such as R-trees [21, 35]. Therefore, the runtime is $O(n^2)$, with linear time range queries and $O(n \log n)$ using a data structure that retrieves data in logarithmic time.

2.1.5 Distance Measures for Comparing Curves

Comparing curves is essential in our application, both within the algorithm itself and for evaluating the results. We considered two metrics: Hausdorff distance and Fréchet distance. The latter was preferred for our use case, as it takes into account the direction and continuity of the curves, which are critical for our problem setting.

The Fréchet distance

The Fréchet distance is a metric designed to quantify the similarity between two polygonal curves. A common analogy used to explain this concept is the scenario of a person walking a dog on a leash. In this example, both the person and the dog traverse their respective polygonal paths from start to finish. While they are allowed to stand still, they are not permitted to move backward along their paths. The Fréchet distance represents the smallest maximum leash length required to allow both to move along their paths.[20]

The Fréchet distance between two continuous curves $f : [0, 1] \to \mathbb{R}^d$ and $g : [0, 1] \to \mathbb{R}^d$ is

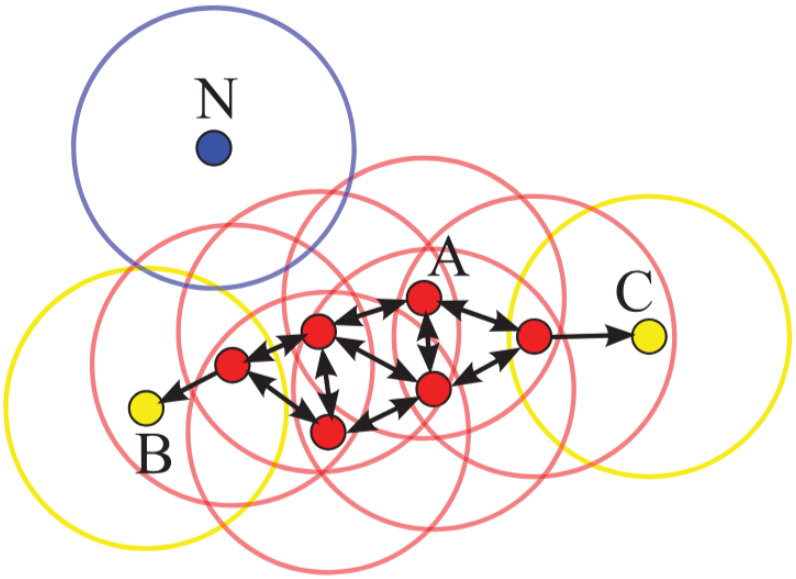


Figure 2.3: DBScan Cluster Model (from [35])

defined as:

$$d_F(P,Q) = \inf_{\substack{\alpha:[0,1]\to[a,a'] \\ \beta:[0,1]\to[b,b']}} \max_{t\in[0,1]} \|f(\alpha(t)) - g(\beta(t))\|.$$

where α, β are continious and non-decreasing reparametrizations, with $\alpha(0) = a$,$\alpha(1) = a'$,$\beta(0) = b$ and $\beta(1) = b'$.

Alt and Godau [6](1992) provided an algorithm to solve the continous Fréchet distance in $O(pq \cdot log(pq))$, where p, q are points of the compared polylines. The discrete Fréchet approximates the continuous Fréchet distance, where only points are considered in the calculation. Eiter and Mannila [20](1994) provided an algorithm to calculate it in $O(pq)$.

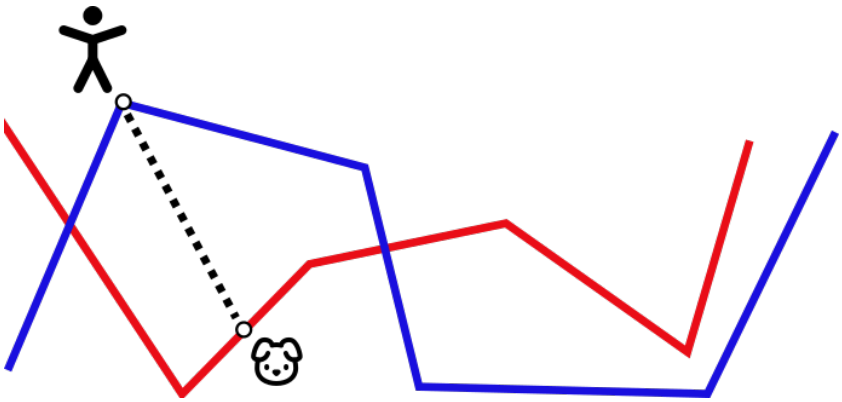


Figure 2.4: "Walking the dog" to showcase the Fréchet distance

The Hausdorff Distance

The Hausdorff distance between two sets A and B is defined as:

$$d_H(A, B) = \max \left\{ \sup_{a \in A} \inf_{b \in B} \|a - b\|, \sup_{b \in B} \inf_{a \in A} \|b - a\| \right\},$$

where $\|a - b\|$ denotes the distance between points $a \in A$ and $b \in B$. The Hausdorff distance measures the greatest distances from a point in one set to the closest point in the other set.

Comparison of Fréchet and Hausdorff Distances

While both distances are used to compare curves, they differ in definition and capture fundamentally different properties:

- The Fréchet distance respects the shape and continuity of curves, making it well-suited for path comparison tasks.
- The Hausdorff distance measures point-wise proximity between curves but ignores the order and direction of traversal.

As shown in Figure 2.5, there are cases where the Hausdorff distance is quite small, but the curves do not resemble each other at all. This is because the Hausdorff distance only accounts for point-to-point distance and does not consider the geometric course, unlike the Fréchet distance. [5]

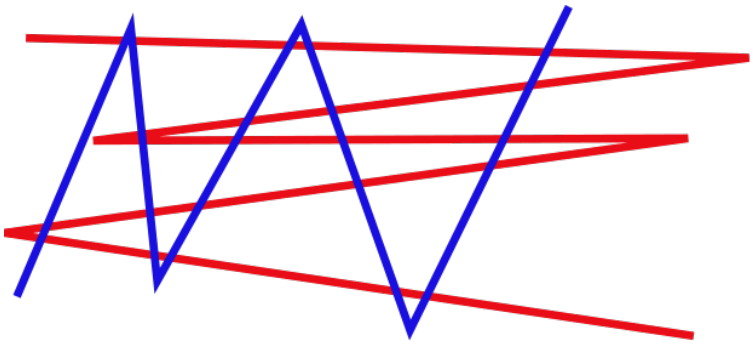


Figure 2.5: Example where Hausdorff distance is not accurate

2.1.6 Free Space Diagram

The concept of Free Space was first introduced by Alt and Godau [5], who defined it as the set of point pairs from curves P and Q, where $(p \in P, q \in Q)$, such that the distance

of p and q is within ϵ. A free space diagram FD_ϵ is then defined as the partition of those point pairs into white and black regions. The white (feasible) points are reachable within ϵ, while the black (infeasible) points are unreachable within ϵ. The white area is called the free space, F_ϵ, and the black area is the remaining space, $FD_\epsilon \setminus F_\epsilon$. For two polylines P and Q, a cell in the free space diagram represents the reachability information of two line segments: one from P and one from Q, as shown in Figure 2.6. In this example, the free space diagram contains a single cell because both P and Q are simple lines, each consisting of only one segment. [5]

Alt and Godau [5] provided an algorithm for solving the decision Problem whether $d_F(P, Q) < \epsilon$ with a runtime of $O(pq * log(pq))$. The algorithm is based on the idea that two curves P and Q are in distance ϵ exactly, if a monotone path exists in F_ϵ from the bottom left to the top right corner. They further used this decision procedure to solve the computation of $d_F(P, Q)$.

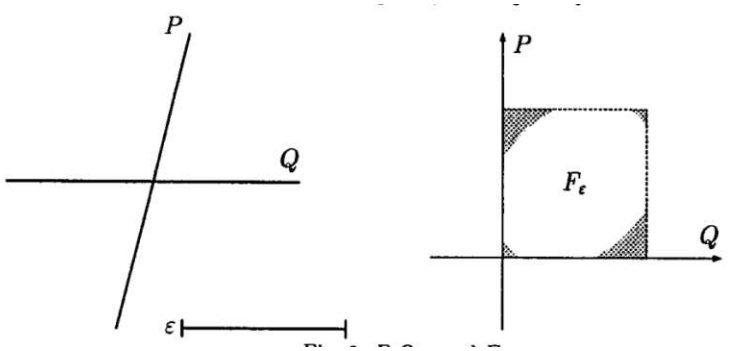


Figure 2.6: Free space diagram created from lines P and Q (from [5])

2.1.7 Partial Curve Matching

Buchin, Buchin, and Wang [16] extended the work of Alt and Godau by producing an algorithm that computes partial curve matching, which means the algorithm yields the interval(s) for which $d_F(P, Q) < \epsilon$ and therefore finds similar parts of two curves. They transform the decision problem of finding a monotone path into a longest-path problem. They find the longest monotone path in the free space diagram, where the weight of the white region is 1, and the weight of the black region is 0.

2.1.8 Matching a Curve to a Graph

Alt, Efrat, Rote, and Wenk [4] studied the similarity of line segments in the plane. Moreover, they proposed an algorithm to match a polyline P to a graph G, with a straight-line embedding in the plane. G is undirected, but each edge between two vertices $i, j \in V$ is represented by two directed edges $(i, j), (j, i) \in E$. A path π in the graph G is the polygonal curve formed by the covered edges.

As shown in Figure 2.7, the free space diagram of the path α and each edge of the graph are glued together according to the adjacency of the graph. A 3D free space surface is created. If a path exists from any left endpoint to any right endpoint in the free space surface, a corresponding path in G that resembles p exists.

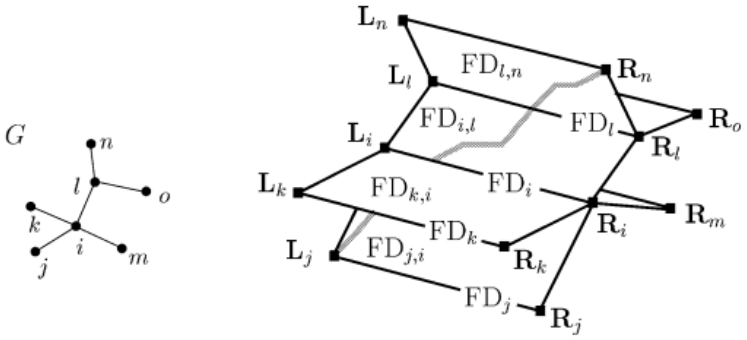


Figure 2.7: 3D free space surface (from [4])

To find a path through the free space surface, first, the decision problem is solved, which determines whether a path exists in G with a fixed ϵ and a Fréchet distance of at most ϵ. They use a dynamic programming technique to reconstruct the path. To find an optimal ϵ-value, a parametric search technique is used.

2.2 State of the art

This section highlights related work in the fields of map construction, graph drawing, map generation, and generalization.

2.2.1 Constructing street networks from GPS trajectories

Similar to our task of building a power line network from individual power lines, Ahmed and Wenk [3] reconstructed a street network from GPS trajectories. The proposed algorithm combines the ideas of partial curve matching [16] and curve-to-graph matching [4].

The algorithm is considered an incremental map construction method because it builds a graph iteratively by inserting lines (GPS trajectories). In each iteration, an input curve l_i is inserted into Graph G_{i-1} from the previous iteration, resulting in an updated graph G_{i+1} [2]. The algorithm accomplishes the task in 3 steps:

1. **Compute Curve-Graph Mapping:** In the first step, the goal is to map the input curve l_i onto the existing graph G_{i-1} while minimizing the unmatched portions of l_i. This involves identifying regions where the curve aligns with the graph (white intervals) and filling the gaps with unmatched regions (black intervals). The process ensures that the mapping is one-sided, prioritizing a maximal match between the curve and the graph. The output of this step is a sequence of black and white intervals along l_i, representing matched and unmatched portions, respectively. [3]

2. **Create/Split Edges and Create Vertices:** In this step, the unmatched portions of the curve l_i identified in Step 1 are integrated into the graph G_{i-1}. For each unmatched interval, new edges are created, and vertices are added or adjusted where necessary. The algorithm splits existing edges to incorporate new vertices and ensures a seamless update of the graph structure. [3]

3. **Compute Minimum-Link Representative Edge:** The final step simplifies the graph representation by replacing matched portions of l_i with a minimal-link curve. This curve retains a bounded Fréchet distance to the original input while reducing the complexity of the geometries. The result is an updated graph, reflecting the input curve's geometries. [3]

2.2.2 Efficient Generation of Geographically Accurate Transit Maps

While most research on metro map drawing focuses on schematic representations, the approach by Bast, Brosi, and Storandt [7] generates geographically accurate maps suitable for use as tiles and overlays in map services. This aligns with our goal of producing a generalized drawing that remains usable in geographic environments such as GIS.

The approach is divided into three phases: line graph construction, line ordering optimization, and rendering. In Figure 2.8, the constructed line graph (left) and the final

drawing (right) are shown. In the line graph notation, each edge records the lines that share a geographic path. [7]

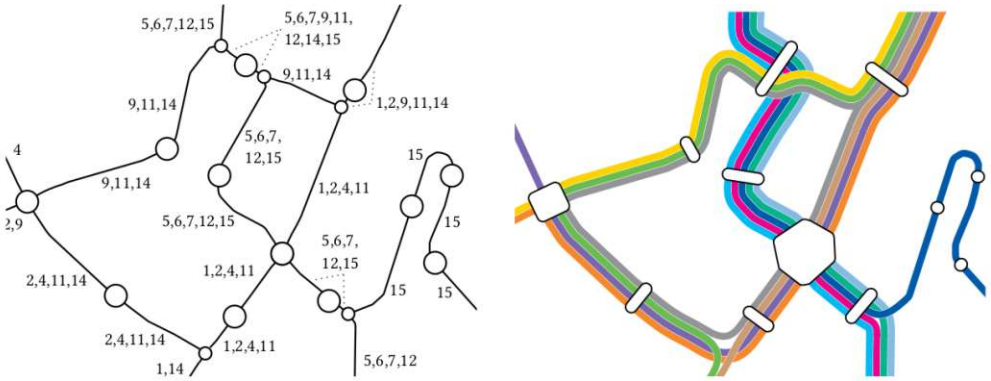


Figure 2.8: Line graph of the metro network and final drawing produced by the loom framework (from [7])

In the first stage, they parse their input data, which is already in the form of a multigraph. They already know the station nodes, but they have to determine additional non-station intersection nodes. As shown in Figure 2.9, the non-station node u' is inserted. They determine the position by traversing the lines in fixed steps and then checking if the points for the lines are still at a certain distance. If the lines diverge, they add an intersection node. [7]

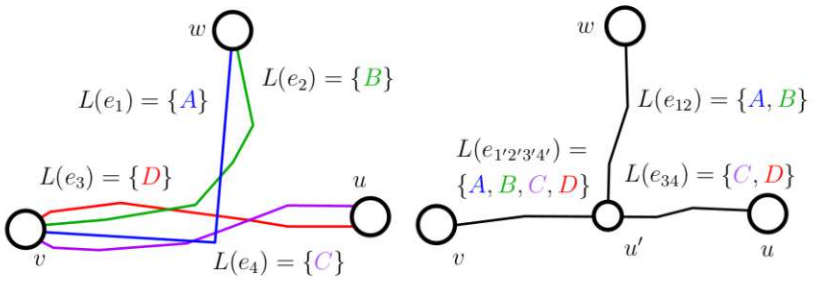


Figure 2.9: Calculation of non-station intersection nodes (from [7])

In the second stage, they optimize the line orderings based on an ILP formulation. [7] However, this can be neglected in our work, as the line orderings should be extracted from the source data.

The last stage handles the rendering of the final map, which is divided into four steps as shown in Figure 2.10. (1) First, they render the individual segments for each line by offsetting the base geometry of the line graph. In the next step (2), they create $deg(v)$ node fronts, and the node area is freed. In the third step (3), they connect all lines in the node areas using Bezier curves. Finally (4), they render the stations.

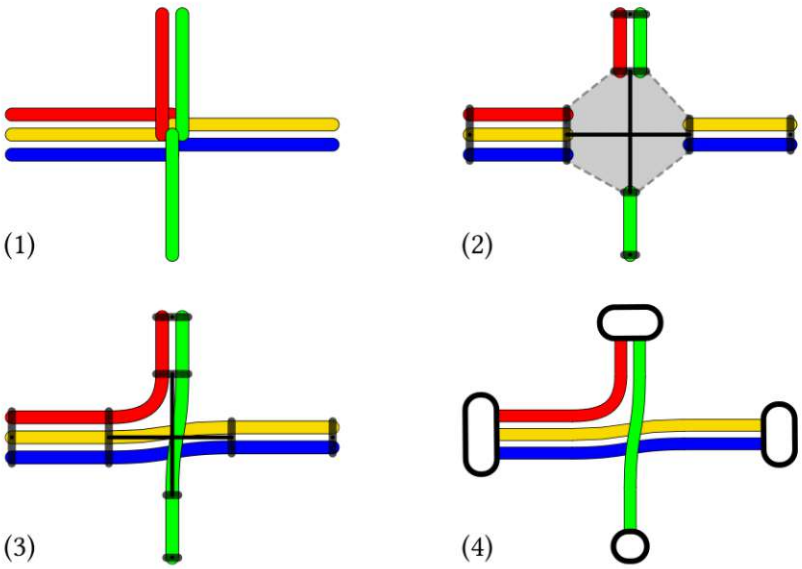


Figure 2.10: 4 steps of the line render phase (from [7])

According to the results, a faster algorithm for constructing the line graph is desirable, as its construction was slower for some instances than solving the ILP step [7]. This suggests that an alternative approach is necessary for our work, given that our network map is an order of magnitude more complex than the average metro map.

2.2.3 Computer-aided Design of Bus route networks

Bus route networks share key structural properties with power grids, where multiple parallel lines follow shared street segments. Sadahiro, Tanabe, Pierre, and Fujii [33] address this with a two-phase algorithm: first computing route orderings per link, then arranging the routes globally. The rendering phase is particularly relevant to our work, as it handles offsetting, aligning, and smoothing overlapping lines in a shared corridor.

In Figure 2.11, a generated bus route map is shown. There are obvious similarities between the shown map and a power network map. Both follow the course of the street network and span multiple parallel lines across segments. The key differences to our work are:

- **No line graph construction** - They assume an already existing line graph, which models stations, bus lines, and which bus lines share the same street segments. Compared to our work, this information has to be extracted from the ground data.
- **Order optimization** - To minimize crossings, shifts, and gaps in their model, they formulate the assignment of routes as a mathematical optimization problem. In

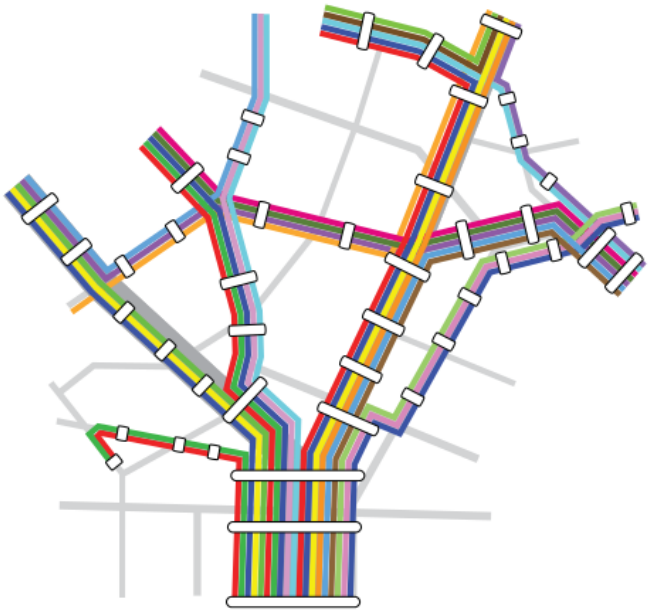


Figure 2.11: Generated bus route map (from [33])

contrast with our work, we want to detect the route orderings from the ground data and keep the ordering intact.

The map layout phase of this work is particularly relevant to our research, as it addresses a problem similar to ours: deriving the final drawing from the line graph and determining the final positions of the routes. The map layout phase first offsets the geometry of the initial segments for each bus line on the given segment. To fix the overlaps, the procedure then continues to shorten and extend the line segments such that they meet at the intersection point. To improve the aesthetics of the drawing, they also insert small segments to fix obtuse angles, as shown in Figure 2.12.

Finally, the authors also reported that manual corrections are usually necessary to resolve some error cases. For example, they provided the problem shown in Figure 2.13. Since the segments share a lot of routes, the available space gets sparse, and the lines start to overlap. To resolve this issue, the lines have to be translated until no more overlap exists.

2.2.4 Large-scale generation of transit maps from OpenStreetMap data

Brosi and Bast [15] recently(2024) presented a global-scale transit map generation pipeline, operating on networks of comparable size and complexity to those found in power grids, making their work particularly relevant to ours. Their approach consists of five steps: (1) extracting network data, (2) constructing the line graph, (3) optimizing line orderings, (4) optionally schematizing the graph, and (5) rendering the final map. The line graph

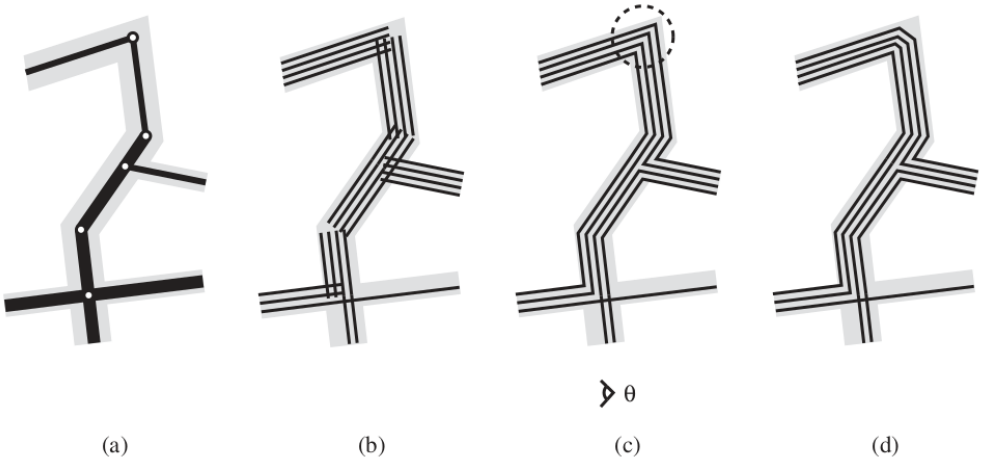


Figure 2.12: Steps of the map layout phase: a) line graph b) initial segments c) segments connected by extension and cutting d) fixed obtuse angles by inserting segments (from [33])

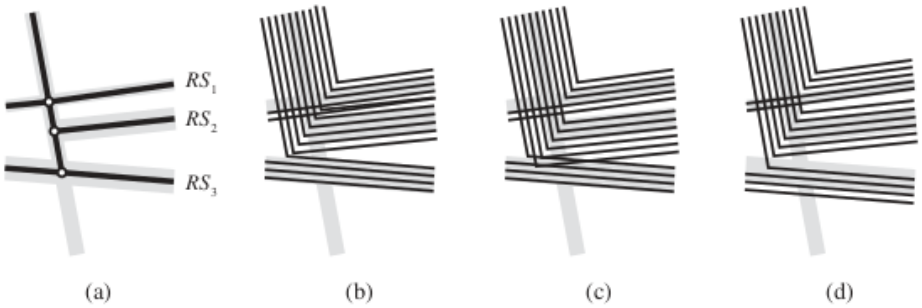


Figure 2.13: Isse with overlapping lines (from [33])

construction step is especially applicable to our work, and their schematization technique offers useful insights for improving visual clarity while preserving topological relationships.

In step 2, the line graph is constructed from the network data, where edges shorter than length d are merged into another edge. As shown in Figure 2.14, they use an incremental map construction approach where edges are inserted one after another into the line graph. The inserted edge is sampled into points with a sampling length l (they used $l = 5$). They then merge the points into the graph with nearby nodes and insert edges where no points are nearby. They repeat the insertion of edges until they reach a threshold value of 0.2% in total edge length change before and after each round.

In Step 4, the network is schematized to create a more visually appealing representation compared to the geographically accurate version. This step is based on the work of [9] and [8]. The method can generate schematized maps in octilinear, hexalinear, and orthoradial styles by mapping the input graph onto a grid graph that reflects the characteristics of

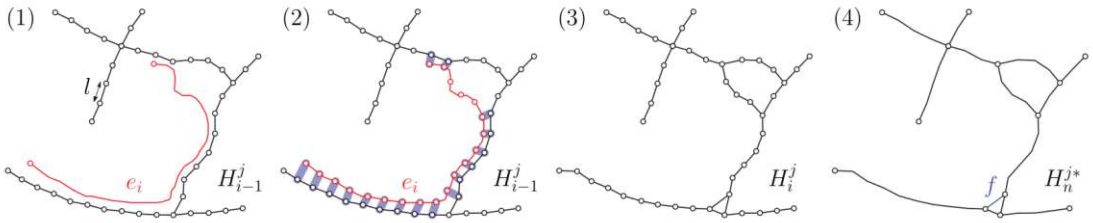


Figure 2.14: Insertion of line into graph (from [15])

the chosen drawing style. To determine the optimal mapping, an ILP-based optimization approach is used, with constraints ensuring key properties such as node-disjoint paths and the preservation of circular edge orderings. However, since solving the ILP formulation becomes computationally infeasible for larger instances, an approximation method is introduced. This method computes short paths in the grid graph for each edge of the original graph, selecting candidate nodes as potential path endpoints.

In the final step, the map is rendered, as shown in Figure 2.15. (1) First, the lines are offset based on the edge polyline. (2) Next, the polygons formed by the line bundles are adjusted to free the node region. (3) The lines are then connected using Bézier curves. (4) Finally, the station markers are rendered.

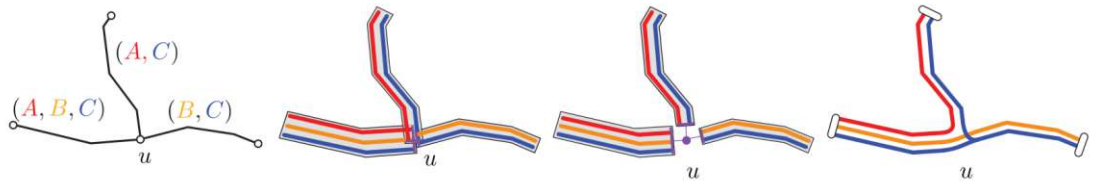


Figure 2.15: Line render process (from [15])

2.2.5 Automatic Generation of Destination Maps

The work of Kopf, Agrawala, Bargeron, Salesin, and Cohen [25] focuses on generating destination maps from street networks. Similar to our approach, they address the merging of parallel road segments and propose a simplification method that preserves the overall network topology.

A destination map highlights only the relevant streets in a local area, aiming to assist users in reaching a specific destination. The approach consists of three phases: (1) road selection, (2) road simplification, and (3) road layout.

The road selection phase identifies the relevant roads for destination maps. However, this step is negligible for our work, as we aim to display the complete network rather than selecting a subset.

In the next step, the roads are simplified by merging parallel lanes and removing ramps, see Figure 2.16. To handle parallel lanes, the method spatially searches for parallel roads with the same street name. As shown in Figure 2.16, nodes with a degree other than two are marked, as they indicate the start or end of streets.

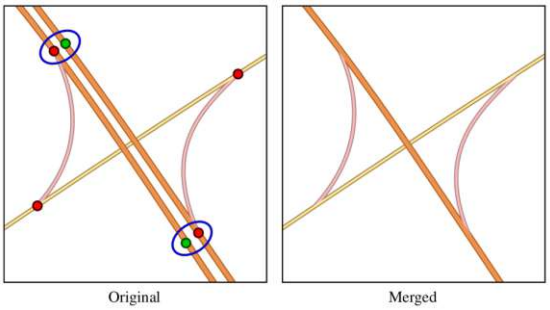


Figure 2.16: Merge parallel roads (from [25])

They follow established design principles that recommend distorting road geometries into straight lines or simple curves. For the simplification process, they use a modified variant of the Ramer-Douglas-Peucker algorithm [31, 18]. Nodes with a degree other than two are fixed, and the algorithm is applied iteratively until no more nodes can be removed. To preserve acute bends, nodes forming angles smaller than a threshold (90 to 115 degrees) are retained.

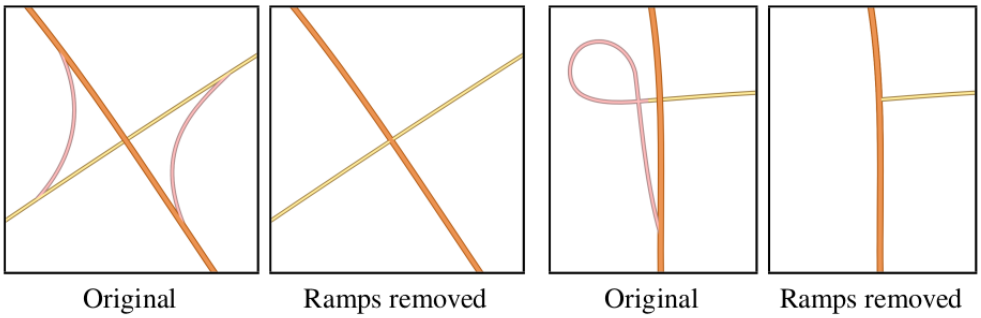


Figure 2.17: Road simplifciation (from [25])

CHAPTER 3

Requirements

As outlined in the introduction, this work employs the Design Science Research (DSR) methodology as defined by Brocke, Hevner, and Maedche [13]. Wiener Netze acts as the key stakeholder, providing the real-world dataset used for the development and evaluation of the algorithm, and serving as the main driver in the evaluation of the findings. The framework by Peffers, Tuunanen, Rothenberger, and Chatterjee[28] describes the process of design science research in 6 steps, as shown in Figure 3.1.

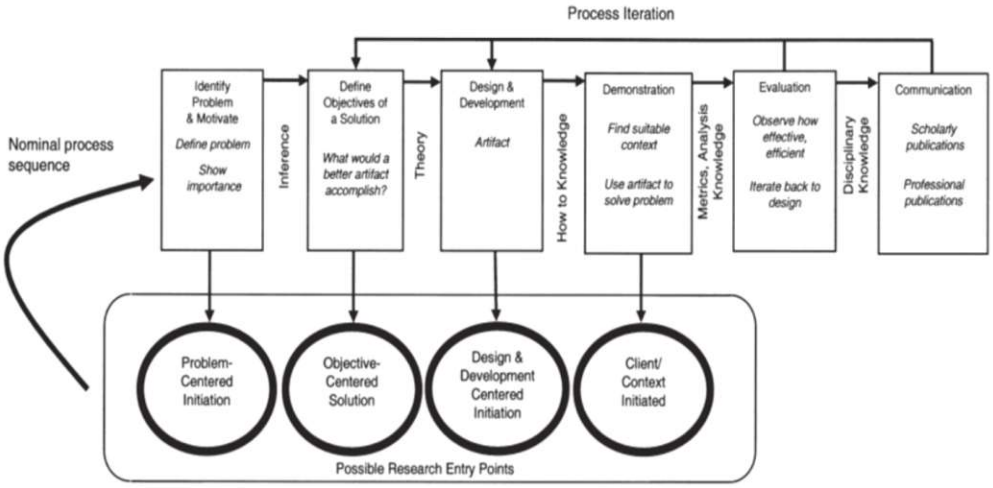


Figure 3.1: Design Science Research Framework, defined by [28] (from [28])

3.1 Requirements Process

The requirements process was framed by the chosen methodology, adhering to the principles and six-step framework described by Brocke, Hevner, and Maedche [13].

Requirements gathering was accomplished through regular unstructured interviews with representatives from Wiener Netze. These interviews were held approximately every two months, totaling five meetings over the course of the project. Since steps 2, 4, and 5 of the DSR cycle involve interaction with the stakeholder, each interview marked the end of one development cycle and the beginning of the next.

The six steps of the DSR cycle applied in this project are summarized as follows:

1. **Problem Identification and Motivation** – The problem was initially introduced during a kick-off meeting with Wiener Netze. An existing legacy solution was available, which provided a reference for the desired outcome. This facilitated communication and helped define the initial requirements and project direction.
2. **Define Objectives for a Solution** – Requirements were gathered during regular meetings with the stakeholders. Based on the established requirements, a literature review was conducted to identify related approaches and technologies. The findings were interpreted and documented in Section 2.2.
3. **Design and Development** – Based on the gathered requirements and research findings, concepts for potential solutions were developed. Prototyping was employed to build minimal viable products (MVPs) and verify their performance against the defined requirements.
4. **Demonstration** – The developed artifacts were tested using the real-world dataset. Five large-scale and fifty small-scale test instances were extracted to allow comparative evaluation.
5. **Evaluation** – The artifacts were applied to solve the test instances. Results were automatically evaluated using established metrics (Hausdorff distance, Fréchet distance, and a crossing score). Additionally, subjective evaluations based on domain expertise were gathered through the interviews with Wiener Netze. Based on these outcomes, new cycles were initiated with updated requirements.
6. **Communication** – Upon completion of the final cycle, the results and insights were documented and communicated through this thesis.

3.2 Legacy Solution

A legacy solution, which was a commercial product, existed prior to this work, but it is no longer maintained. The reported runtime of the legacy tool was approximately 48 hours. Although the output quality was generally acceptable, manual conflict resolution

was still required after each automated run. The main reason Wiener Netze aimed to replace the system was its obsolete and unmaintained state. Consequently, its output serves as a reference point for this project. The source code was not available.

3.3 Cycle Overview

This section provides an overview of the development cycles and the challenges addressed in each phase.

Cycle 0 (Preparation Phase)

Requirement Refinement The project began with a kickoff meeting to discuss initial requirements. For internal compatibility with developer skills, the new solution should use Python as a programming language **R1**. Further, the solution should produce a more readable generalized output drawing from the input **R2**. To adjust the scale according to preference, the output scale should be a configurable parameter **R3**.

Design and Implementation With these requirements established, a literature review was conducted to gather initial background knowledge. An attempt was made to generate a synthetic dataset for early testing. However, the quality of the generated data did not meet the standards set by the real-world dataset.

As the first result, the general framework for the algorithm was developed conceptually. The idea was first to detect parallel lines to build the skeleton of the network and then use this to split up the parallel power lines using the configurable scale parameter, to fulfill **R3**.

Cycle 1 (until 25.07.2024)

Requirement Refinement The real-world dataset and the output of the legacy solution were provided, which created the requirements to accept geodatabases (with shapefiles for line and point layers) as input **R4**, and to handle the data confidentially **R5**. Further, the requirements to preserve the original line order across segments **R9**, and targeting a scale of 1:2000 **R6** have been defined.

Design and Implementation As a first step to fulfill **R4**, a method to read data from a geodatabase was developed. A line graph construction phase was developed, inspired by the work of Bast, Brosi, and Storandt [7]. It detects and merges parallel lines to support subsequent generalization steps.

However, due to the dataset's large size compared to typical metro networks, performance was poor: even small subsets required several minutes of computation. As a result, this approach was discarded. Although not yet formalized, this would have violated the runtime constraint later defined as **R12**.

A point clustering approach based on [19] was also evaluated. While faster, it failed to preserve line order in dense regions, conflicting with **R9**. Nonetheless, a first version of the rendering component was implemented to assess output readability, supporting initial progress toward **R2**.

Cycle 2 (until 25.09.2024)

Requirement Refinement With the first visual results, in a follow-up meeting, the requirement to incorporate network elements, such as transformers, into the map was added. These occupy a fixed size of 2x6 meters in the final drawing, **R7**. Additionally, support for both line and point data was formalized as a requirement **R10** due to the input dataset's composition. While lines describe the power lines of the network, point data describes components of the network, which should also be moved according to the generalization process.

Design and Implementation Although the concept of constructing a graph from the network data conceptually supported the idea of embedding point data as nodes into the algorithm, the work on **R7** and **R10** had to be postponed, since the line graph construction results were not satisfying. Following, work continued on line graph construction, towards **R2** and **R9**. Track insertion algorithms showed promise, as they operated on graphs capable of storing point data at nodes and supported the insertion of complete line segments. These approaches yielded better results than the previously evaluated point clustering methods [1].

Cycle 3 (until 25.11.2024)

Requirement Refinement A new requirement was introduced to allow the selective removal of certain network features, such as housing connections **R13**. Additionally, the requirement to prioritize the readability of the output drawing over strict geographic accuracy was formalized **R8**.

Design and Implementation The line graph construction was implemented, based on the method proposed by [3]. Since this method focuses solely on merging and does not handle line splitting, a custom extension was developed to preserve the original order of line segments, addressing requirement **R9**.

Cycle 4 (until 25.01.2025)

Requirement Refinement Requirement **R11** replaced **R7** to allow flexible space allocation, not only for transformators, but all types of elements in the network, we should be configurable. Additional requirements for improved runtime **R12**(below 48 hours), and selective feature removal **R13** were also established.

Design and Implementation Special case handling was added to improve both network integrity and visual output. This included preprocessing, edge case management, specific handling for housing connections, and better alignment of straight-line segments. The handling of housing connections also enabled their inclusion in the output, contributing to progress on requirement **R10**.

Cycle 5 (until 05.05.2025)

Requirement Refinement No further requirements were established.

Design and Implementation Final refinements to the rendering component were completed, accompanied by a detailed evaluation using quantitative metrics. Since overall significant effort was dedicated to fulfilling requirements **R2** and **R9**, which were essential for enabling generalization. Requirements **R7**, **R11**, and **R13** could not be fully implemented and remain open items for future development.

3.4 Requirements List

The final list of requirements derived through the DSR process is as follows:

R1 The solution must be implemented in Python.

R2 The original drawing should be generalized to improve readability at smaller scales.

R3 The output scale must be configurable; the algorithm should support generalized drawings for different scales.

R4 The algorithm should accept a Geodatabase as input, where the individual layers are shapefiles: line and point layers.

R5 The network data is confidential, and it must not be transferred to third parties or external services. Further, the thesis must not disclose critical infrastructure details.

R6 The base scale is approximately 1:200, and the target scale is approximately 1:2000 in a map view.

R7 Transformers must be considered in the final drawing with a fixed size of 2x6 meters.

R8 Readability is prioritized over strict geographic accuracy in the output drawing.

R9 The ordering of lines along segments should be preserved as in the original drawing - optimization of order is not required.

R10 The algorithm must support both line and point data.

R11 Different elements require configurable space allocation in the final drawing.

R12 The total runtime should be equal to or faster than the legacy solution (reported at 48 hours).

R13 The solution must support selective removal of certain features, such as housing connections, if necessary.

CHAPTER 4

Algorithm

This chapter introduces the algorithm designed to solve the problem outlined in Chapter 1. The power lines are arranged in cable trays, forming bundles of parallel segments (Figure 4.1). By analyzing branching points, we can determine junctions in the network, allowing us to construct its underlying skeleton, which has the technical meaning of the cable tray network. From this network, we then split up the single cables with the preferred width (**R3**) to generate a drawing that is readable at a larger scale, fulfilling **R2**.

The algorithm is divided into three phases. First, the data is preprocessed by cleaning and sorting the input lines. In the second phase, parallel line segments are detected and merged to construct the skeleton network from the input data. In the context of our algorithm, we refer to this skeleton as the line graph G_{line}, Figure 4.1. Finally, edges of the line graph are split again and geometrically offset based on the target line spacing l to create the final drawing, Figure 4.1.

4.1 Technical Implementation Detail

As defined by requirement **R1**, the algorithm is implemented in Python 3.13.1. The algorithm processes the data locally to conform to the requirement **R5**. The implementation relies on several dependencies, listed here:

- **rustworkx** - A high-performance graph library [38] used for network representation and graph algorithms.
- **GDAL** - Used for reading and writing geospatial data.
- **rtree** - Provides spatial indexing to enable fast geographic lookups and nearest neighbor searches.

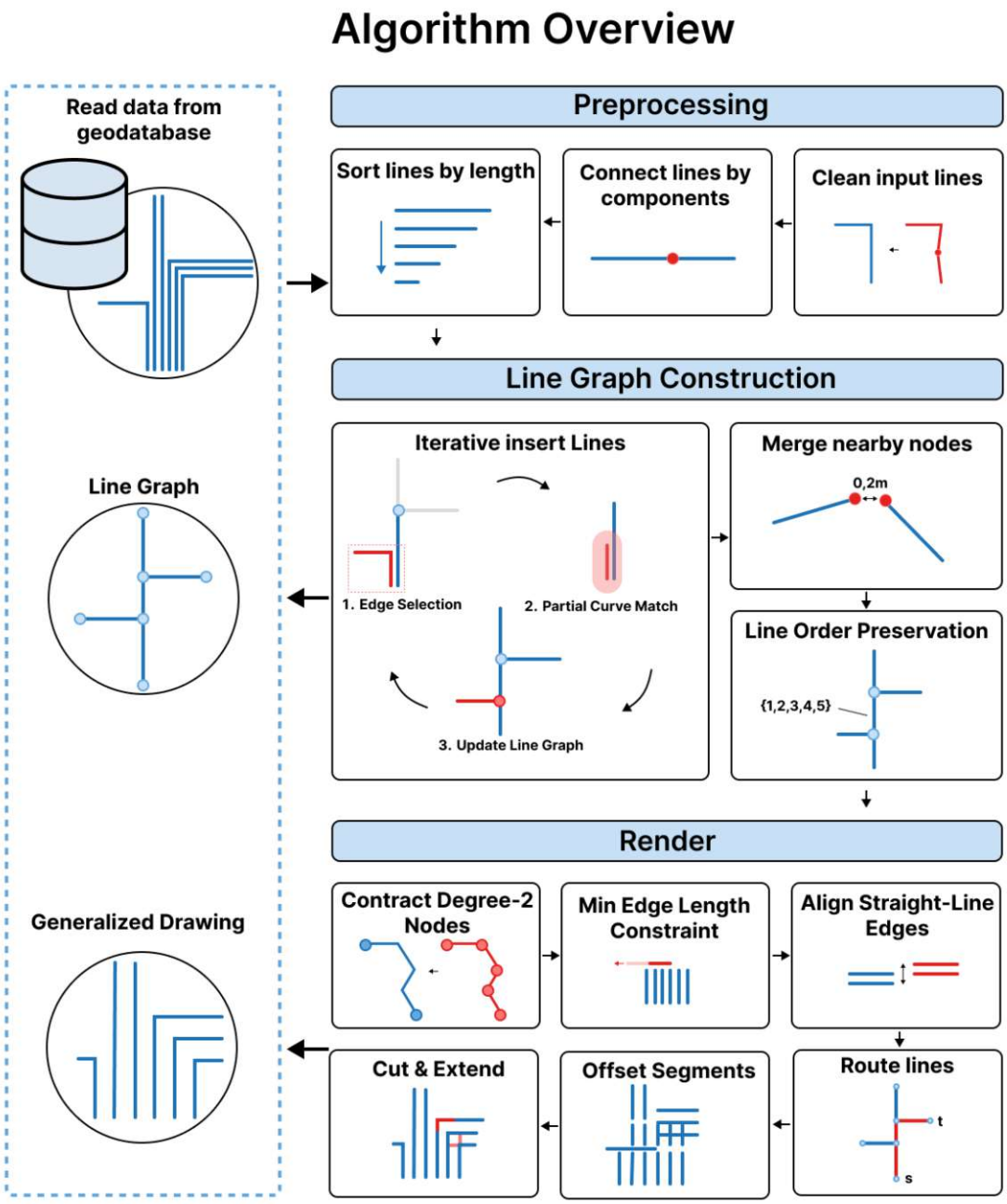


Figure 4.1: Overview of the algorithm pipeline and its main components

- **shapely** - Used for geometric operations as intersection testing, line offsetting, and other transformations.
- **numpy** - For fast numerical operations and convenience methods.
- **scikit-learn** - Used for clustering via the DBSCAN algorithm.
- **heapq** - Not a dependency, but a Python module providing a Minimum-Heap data structure, used to implement a priority Queue.
- **similaritymeasures** - Not used in the algorithm itself, only in the evaluation.
- **pandas** - Not used in the algorithm itself, only in the evaluation.

4.2 Input Data

Before processing, the input data must be loaded from a Geodatabase, an ESRI-specific file format that stores geo-related information, as defined by requirement **R4**. The primary input consists of line and point data, which are parsed to construct a graph representation of the power grid network. Each geographic feature has a unique *featureId*, which will be used through the algorithm to reference and identify features.

4.2.1 Line data

The primary dataset consists of a collection of *LineStrings*, where each LineString represents an individual power line. A LineString is defined as an ordered sequence of geographic points that form a continuous line in 2D space. In the context of power line networks, these LineStrings define the spatial paths of power transmission between network elements. We will refer to these line features as *lines* for simplicity.

4.2.2 Point Data

Although not essential for the line graph construction, the dataset also includes *point features* representing specific network elements. As defined by requirement **R10**, point data should also be transformed according to the generalization process. These points can serve as useful reference locations in certain situations, such as aiding in the construction of the line graph by marking line origins or intersection points. The dataset comprises multiple point datasets, each corresponding to different types of network elements, including various components of the power grid infrastructure, such as transformer stations, residential connections, and light poles.

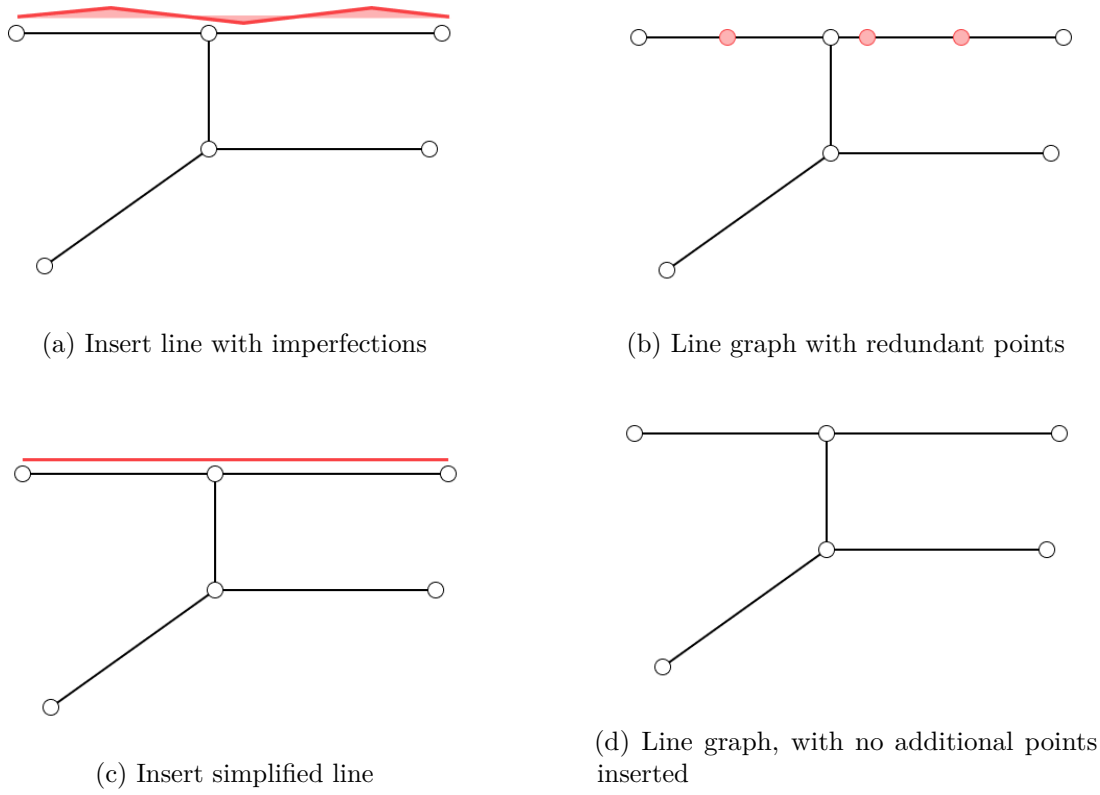
4.3 Preprocessing

Before the main procedure starts, the input lines are simplified to prevent problems in the construction phase and decrease the runtime by reducing the number of comparisons.

As shown in Figure 4.2, when the inserted line has imperfections, where consecutive points are nearly colinear, redundant points are created. The issues could be resolved during the construction phase, but with preprocessing, we have a rather cheap solution to fix the problem beforehand.

To mitigate these issues early on, the line segments are cleaned, which involves:

- **Removing duplicate coordinates:** For a given set of coordinates $\{p_1, p_2, \ldots, p_n\}$, any pair $p_i = p_j$ (where $i \neq j$) is removed, ensuring that each coordinate is unique.
- **Eliminating collinear points:** Consider three consecutive points $p_1 = (x_1, y_1)$, $p_2 = (x_2, y_2)$, and $p_3 = (x_3, y_3)$ in a line string. These points are collinear if the cross product of the direction vectors $\mathbf{v}_1 = (x_2 - x_1, y_2 - y_1)$ and $\mathbf{v}_2 = (x_3 - x_2, y_3 - y_2)$ is zero. In practice, we apply a threshold angle (in degrees) to account for nearly collinear points. Since collinear points lie on the same straight line, the middle point p_2 can be removed to simplify the geometry.



(a) Insert line with imperfections

(b) Line graph with redundant points

(c) Insert simplified line

(d) Line graph, with no additional points inserted

Figure 4.2: Comparison of the insertion of the original and simplified line

4.4 Construction of the Line Graph

In the construction phase, we build the line graph from the list of input lines. The approach is an incremental algorithm based on the work of Ahmed and Wenk [3]. The key idea is to build the line graph G_{line} incrementally by processing each line successively, ordered by length, and inserting it into the graph. Matched and unmatched portions of the line are calculated, using the concept of free space. To decide which portions are inserted, which lead to updates of current edges in the graph. The approach is divided into 3 steps: (1) Edge selection, (2) Match calculation, and Update of the line graph (3).

Storing intermediate results in a graph is beneficial in many ways. Since nodes represent geographic locations, the integration of point and line data, as stated in Requirement **R10**, is straightforward. Moreover, the flexible structure also provides support for displacement strategies, needed to accomplish **R7** or **R11**.

Figure 4.3 shows one iteration, which corresponds to the insertion of one l_i into G_{line}. For the matched portion, edge (u, v), the structure of G_{line} remains unchanged, and we extend the edge metadata with the line's *featureId* (3). For edge (v, w), the line partially matches l_i, so we create a split node x. The matched portion is then updated with the corresponding *featureId*. Finally, the unmatched portion is inserted as new edges (x, y) and (y, z). This process is repeated until all lines are processed and the line graph is completely built.

The resulting graph is undirected, and each node is assigned a geographic location in the form of coordinates. Consequently, any arbitrary path within the graph can be transformed into a geometric line by concatenating the coordinates of its nodes.

(a) l_i is about to be inserted in G_{line} (b) Edges are added to G_{line}

Figure 4.3: Line graph construction stage

We begin by sorting the lines in descending order based on their geometric length to optimize runtime efficiency. This approach ensures that the longest line is inserted first,

minimizing conflicts and reducing the number of required comparisons. As shorter lines are processed later, the queried areas remain small, thereby limiting the total number of edges that need to be matched against each line.

Algorithm 4.1: Calculate line graph

Input: Array L of size n
Output: Line graph G_{line}

```
1  sort L by length;
2  G_line ← Empty graph;
3  for i ← 1 to n do
4      (l_i, fid) ← L[i];
5      E_nearby ← findNearbyEdges(l_i, G_line);
6      M ← getPartialCurveMatching(l_i, E_nearby);
7      foreach m in M do
8          (lineStart, lineEnd, edgeStart, edgeEnd, v, w) ← m;
9          if m is white interval then
10             Update existing edge (v, w) in G_line with fid;
11             segment ← Matched portion of original line l_i,
12             from lineStart to lineEnd;
13             Add segment to the edges assigned segment List;
14         end
15         else if m is black interval then
16             Add a new edge to G_line;
17             Assign segment to edge (v, w);
18         end
19     end
20 end
21 foreach (v, w) in G_line.Edges do
22     L_segments ← Stored list of matched segments for (v, w), from last step;
23     Find geoemtric order of L_segments;
24     Assign a List of fid's according to segment order;
25     Delete L_segments for (v, w);
26 end
27 return G_line;
```

4.4.1 Step 1: Edge selection

We process the lines by building $G_i = (E_i, V_i)$, for each iteration, where the line l_i is inserted into G_i to build G_{i+1}. To avoid matching each line with each edge in the graph, we select a subset of edges $E_{nearby} \subseteq E_i$ close to the inserted line l. A geospatial index stores the graph's current edges and performs queries to select nearby edges. As shown in Figure 4.4, we buffer l_i with the track width t. All edges inside the resulting area are selected for comparison in the next stage. Conceptually, we operate on a subgraph of G_i

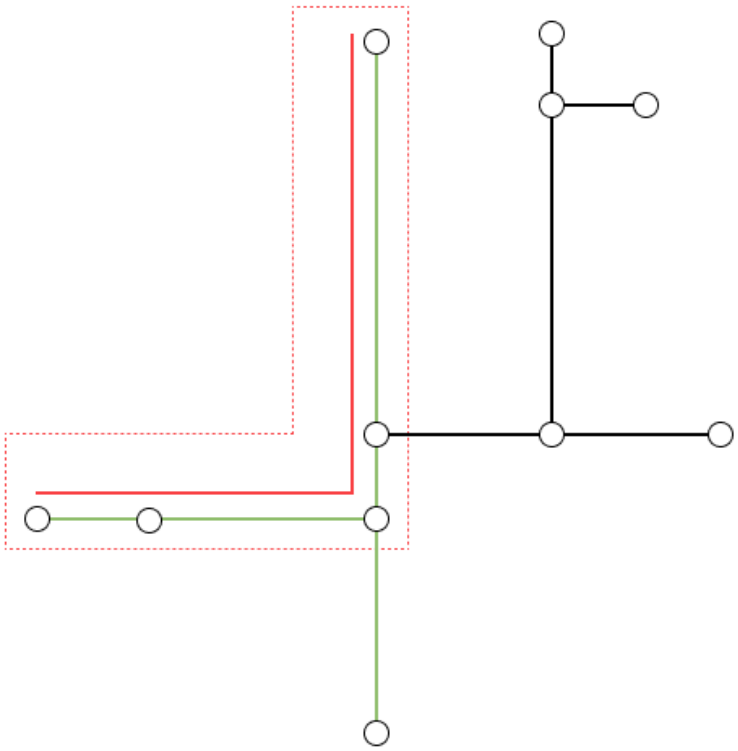


Figure 4.4: Edge selection for l_i to compute matches

but do not calculate it.

4.4.2 Step 2: Calculating Partial Matches

This step is based on the work of Ahmed and Wenk [3]. We calculate the matched portions of each inserted line. These matched portions are referred to as *white intervals*, which align with the concept of free space, as introduced in Subsection 2.2.1. The white intervals I are represented as a list of tuples in the form of $(\text{edge\_start}, \text{edge\_end}, \text{line\_start}, \text{line\_end})$, where the following condition holds:

$$\text{line\_start}_{i-1} < \text{line\_end}_{i-1} < \text{line\_start}_i < \text{line\_end}_i$$

Therefore, we receive an ordered list of matched portions, where the gaps are filled with *black intervals* (unmatched portions).

To perform the actual calculations, we always calculate them between straight line segments of an edge and a segment of l_i. Conceptually, we calculate the border of a free space cell, as shown in Figure 2.6, where ϵ is set to the track width parameter t.

To calculate the matches list M, we use Algorithm 4.2. We begin by calculating the first interval for each selected edge by searching through the segments of l_i to find the next white interval. We stop searching as soon as we encounter the next black interval, which indicates that the current white interval has ended. Each found interval is then inserted into a priority queue, ordered by *line_start* and *line_end*. If no white interval is found for an edge, it means that the edge has no matching portion and will not be considered for this l_i.

After the preparation step, we start with the selection of intervals. We pop values from the priority queue and try to find matches for l_i, which consists of p line segments. We keep track of the current pointer, which is 0 initially. When we add an interval, the pointer is set to *line_end* of the interval. The following rules have to be considered:

- When a *line_end* is smaller than the pointer, we drop the interval, which means we already covered this portion of the l_i.
- When *line_start* overlaps with the current pointer, we set the interval start to the pointer value to prevent overlapping intervals.
- When the value of *line_start* exceeds the stored pointer, indicating a gap, we fill this gap with a black interval. Since we process the intervals in the order determined by the priority queue, we can be certain that such an interval does not exist.

4.4.3 Step 3: Updating the line graph

After calculating the List of Matches L_i, which gives us the partial Matches from line l_i and Graph G_i, we can update G_i. We proceed in two steps:

- For **white intervals**, we update the information associated with the matched edge. If an edge is matched partially, we need to split the edge by inserting a new node. When an edge is split, only the matched part of the edge is updated.
- For **black intervals**, we insert new edges. Neighboring white intervals determine the nodes of G_i where the new edge will be inserted. A black interval will always have a neighboring white interval, otherwise, it's the only interval in the list, and then we insert the edge as a detached component in G_i.

Algorithm 4.2: Calculate Partial Matches

Input: Local Graph G_i=(E_i, V_i)
Input: Line l_i
Output: List of Intervals L

```
1  pq ← [];
2  foreach e in E_i do
3  |   interval_next ← getNextInterval(e, p_i[0]);
4  |   if interval_next is not null then
5  |   |   Add interval_next to pq;
6  |   end
7  end
8  L ← [];
9  line_pointer ← 0;
10 while pq not empty do
11 |   i ← pq.pop();
12 |   if line_start_i ≤ line_pointer then
13 |   |   line_start_i ← line_pointer;
14 |   end
15 |   else
16 |   |   Add black interval to L;
17 |   end
18 |   interval_next ← getNextInterval(e, p_i[line_pointer]);
19 |   if interval_next is not null then
20 |   |   Add interval_next to pq;
21 |   end
22 end
23 return G_line;
```

Algorithm 4.3: Line Graph Update

Input: Line Graph $G_i = (E_i, V_i)$
Input: Array M of size n
foreach *white* ***in*** M **do**
  $(lineStart, lineEnd, edgeStart, edgeEnd, v, w) \leftarrow white$;
  if $edge\_start > 0$ **then**
    $coordinates \leftarrow interpolatePoint((v, w), edge\_start)$;
    Add split node u at $coordinates$;
    Remove edge (v, w);
    Add edges (v, u) and (u, w);
  end
  if $edge\_end < 1$ **then**
    $coordinates \leftarrow interpolatePoint((v, w), edge\_end)$;
    Add split node u at $coordinates$;
    Remove edge (v, w);
    Add edges (v, u) and (u, w);
  end
  Update matched edge (v, w) or (v, u), (u, w);
end
foreach *black* ***in*** M **do**
  $(lineStart, lineEnd) \leftarrow black$;
  $x, y \leftarrow$ empty, empty;
  if *black is first* ***or*** *previous interval is black* **then**
    $x \leftarrow$ Insert new node at $interpolateLine(l_i, lineStart)$;
  end
  else
    $x \leftarrow w$ from previous interval matched edge (v, w);
  end
  if *black is last interval black* ***or*** *next interval is black* **then**
    $y \leftarrow$ Insert new node at $interpolateLine(l_i, lineEnd)$;
  end
  else
    $y \leftarrow v$ from next interval matched edge (v, w);
  end
  Add new edge (x, y) to G_i;
end
return G_{line};

4.4.4 Preservation of line orders

Another critical part of the solution is preserving each segment's line orderings, as defined by requirement **R9**. The individual power lines should have the exact same order in the output as in the input drawing.

While the first step infers the network topology, we additionally need to record the current line orderings to restore the sequence of matched segments later in the rendering phase. We store the matched line segments for each matched portion and the *featureId*.

After the line graph is built, we determine the order of the matched portion within each edge. This involves comparing each segment with the representative segment (edge) and sorting them based on their intersection points with a line perpendicular to the representative segment. Each segment is projected onto the imaginary line, and then the ordering is calculated based on the intersections alongside the line, as shown in Figure 4.5. As a result, we have defined for each edge of G_{line} the correct order of segments, which is stored as a list of *featureIds*. Consequently, the undirected graph is assigned an implicit direction, since the order is only valid if viewed from an edge direction.

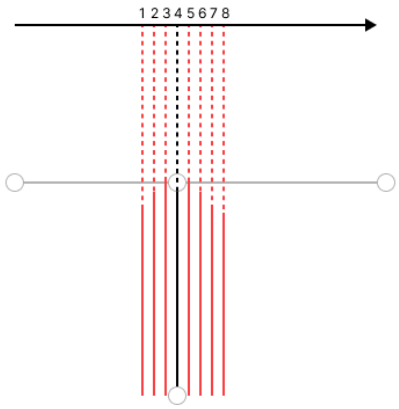
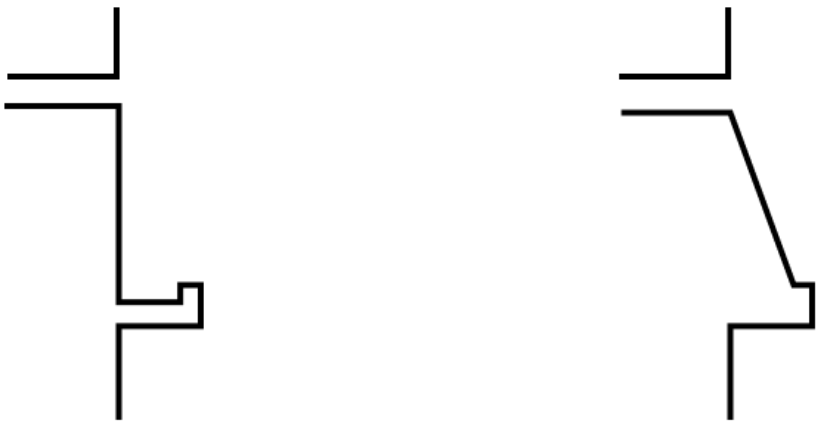


Figure 4.5: Calculation of line order

4.4.5 Prevent unrelated line matches

A common problem during line graph construction was mismatches between unrelated line segments, as shown in Figure 4.6. The short segment of the L-shaped feature was incorrectly matched with a longer line segment running alongside the street path, resulting in an ambiguous connection.

These L-shaped features represent housing connections, whose locations are specified in the source dataset (see Section 4.2). To resolve this issue, we mark these nodes as housing connections. Only edges originating from the same endpoint type are considered valid matches during the matching process.



(a) Original (b) Line graph

Figure 4.6: Problem with unrelated line matches

4.4.6 Straight line interuptions

Another issue arose from lines in the input data that did not connect to the network's endpoints but instead terminated mid-segment while extending straight into another line (see Figure 4.7). This caused the line graph to be disconnected, even though the original data suggested connectivity.

To address this, we utilized an additional point dataset representing components, as described in Section 4.2. These components are consistently positioned between such discontinuous line segments (see Figure 4.7).

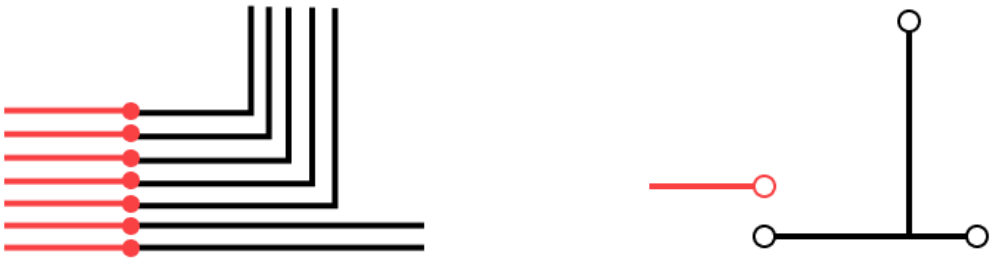
Leveraging this property, we merged lines at the component positions to create continuous lines that properly connect the network's sources and sinks. An alternative approach would have been to map the components to the original line endpoints and incorporate them during the matching process. However, for the sake of a simpler line graph and geometry, we opted for the former.

The caveat is that the components must be reinserted, and the lines must be split back into their original segments after generating the final output.

4.4.7 Conflicting junctions and nodes

This postprocessing step handles ambiguities that arise during the line graph construction. First, we merge nearby points within 0.2m. We use DBScan to cluster the points and then merge the nodes of a cluster into one new node while contracting all neighboring edges.

To fix junctions where the algorithm did not record each track correctly, we apply a



(a) Original (b) Line graph

Figure 4.7: Problem with interrupted lines

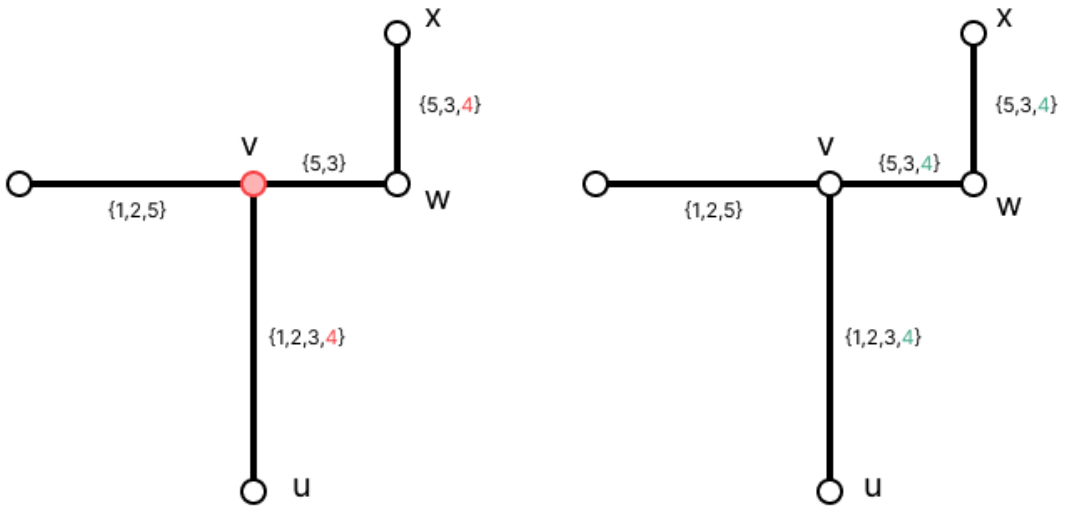
procedure to restore ambiguous junctions. As shown in Figure 4.8 (a), line 4 is interrupted because the algorithm failed to label the edge (v, w) correctly. This will cause a problem in the render phase since we cannot draw a continuous path. By definition, each power line has to be a continuous path. To restore the missing link, we observe each node $v \in V_{\text{line}}$ such that $\deg(v) > 2$. For each v, we look at all incident edges of v to detect conflicts at junction points. Since those nodes are neither start nor endpoints of the network, we know each power line has to occur exactly two times in all adjacent edge lists. In Figure 4.8, node v has a conflict since the number 4 only occurs in one edge. The edge (u, v) is called the conflicting edge.

To restore the path, we must select the edge where the *featureId* is missing, which is (v, w) in our example. We do not know this initially because it could be any edge adjacent to v, not the conflicting edge itself. Therefore, we again look at all adjacent edges and check along the paths of the edge until we find an edge with the missing *featureId*, where paths are a list of connected edges with degree 2. If we could find such a path, the originating edge would be our selection where we add the missing *featureId*. Otherwise, we can not solve the conflict. In the example, we find such a path since edge (w, x) has *featureId* 4, and we modify (v, w) to restore the path, Figure 4.8 (b).

4.4.8 Parallelization

Since the approach for constructing the line graph operates only within the local neighborhood, parallelization could be introduced to accelerate computations. The following approach outlines a possible implementation:

1. Divide the total area into equal-sized rectangular sub-areas.



(a) Original (b) Line graph

Figure 4.8: Problem with invalid crossings

2. Create buckets for each *sub-area*. Assign each line to a single *sub-area* whenever possible; if a line spans multiple sub-areas, place it in an additional *multi-area* bucket.

3. Execute the algorithm in parallel for each *sub-area* bucket, excluding the *multi-area* bucket.

4. Merge all sub-areas into a single line graph with multiple disconnected components.

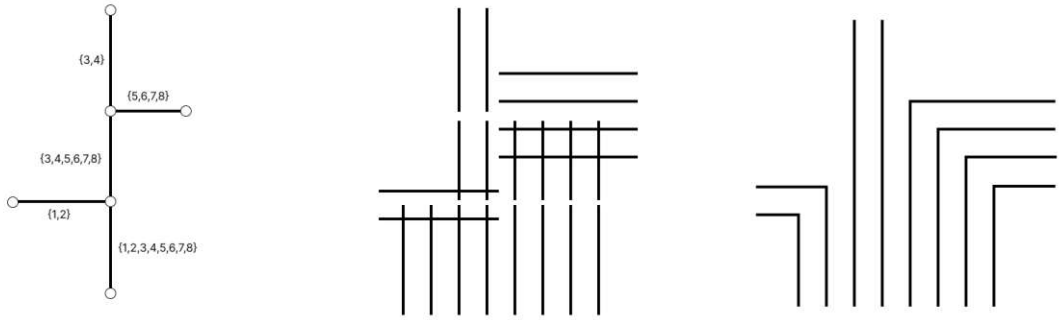
5. In the final step, process the lines in the *multi-area* bucket. These lines often include longer ones that span multiple regions and thus play a key role in connecting components. Whether sorting these lines by length still improves performance in the parallel setting remains an open question and should be evaluated.

4.5 Rendering

In this step, we generate the final drawing. First, we prepare the line graph by enforcing an optional minimum perpendicular edge length constraint and aligning straight-line edges. To construct paths from the line graph, we first simplify the structure of the graph by contracting nodes of degree 2.

Next, we compute the shortest paths between the start and destination points for each line. This step compensates for potential mislabeling during line graph construction; without it, missing labels could prevent the line from being rendered altogether. However, this workaround is fragile, as shown later. A more sophisticated solution is needed to reliably handle labeling gaps.

Using these paths, we construct each line by offsetting it according to its track number and then combining them geometrically to render the final generalized lines. The key idea for the rendering process works as described by Sadahiro, Tanabe, Pierre, and Fujii [33]. From our line graph Figure 4.9a, we offset our geometries to lay out the basic paths, Figure 4.9b. Finally, we receive the final drawing by connecting the paths through cutting and extending the line segments. Figure 4.9c



(a) Line Graph (b) Offset line segments (c) Generalized

Figure 4.9: Render phase overview

4.5.1 Enforce minimum perpendicular edge lengths

When we observe the offset tracks of an edge and the edges have more than one track, we see that each of those edges spans a rectangle, with the length equal to the respective edge length and the width equal to $(n-1)*t$, where n is the number of tracks and t the track width, see Figure 4.10.

A problem arises in the rendering process when a perpendicular edge has a length smaller than $(n-1)*t/2$, as shown in Figure 4.11a. If we look at the bounding rectangle of the two edges, we can see that the rectangle of the green edge is contained in the rectangle of the blue one, see Figure 4.11b. When we want to apply our procedure to connect line

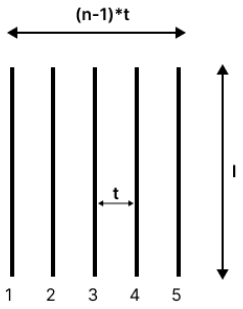


Figure 4.10: Rectangle spanned by edge

segments, it will fail since the green segment of line 5 is only extended in the direction pointing inwards, see Figure 4.11c.

To resolve the problem, we ensure that every perpendicular edge has a length bigger than $(n-1) * t/2$, where n is the track count of the perpendicular edge. In our example, we simply translate the endpoint of the green edge to ensure the minimum edge length is reached. Since the edge is a sink, no further constraints are applied. However, in most cases, additional features will be connected to the edge, which have to be translated and checked with other constraints.

In our solution, only simple cases are handled. There is a need to handle more complex cases:

- The trivial case described above, when the perpendicular edge is a sink edge.
- For an edge (u, v), when the Perpendicular edge (v, w) is followed by a straight aligned edge (w, x) and the straight edge is longer than the space needed by edge (v, w), the minimum edge length is also respected in the calculation.
- For an edge (u, v), if u is a sink node and there are two perpendicular edges, (v, w) and (v, x), then we can move the edge (u, v) along the direction of the other two edges. In this case, we have to ensure both edges (v, w) and (v, x) pass the minimum edge length constraint.

4.5.2 Contract degree 2 nodes

To reduce the complexity of the path-finding step, we decrease the number of edges by removing degree-2 nodes. Specifically, we identify all degree-2 paths, which means:

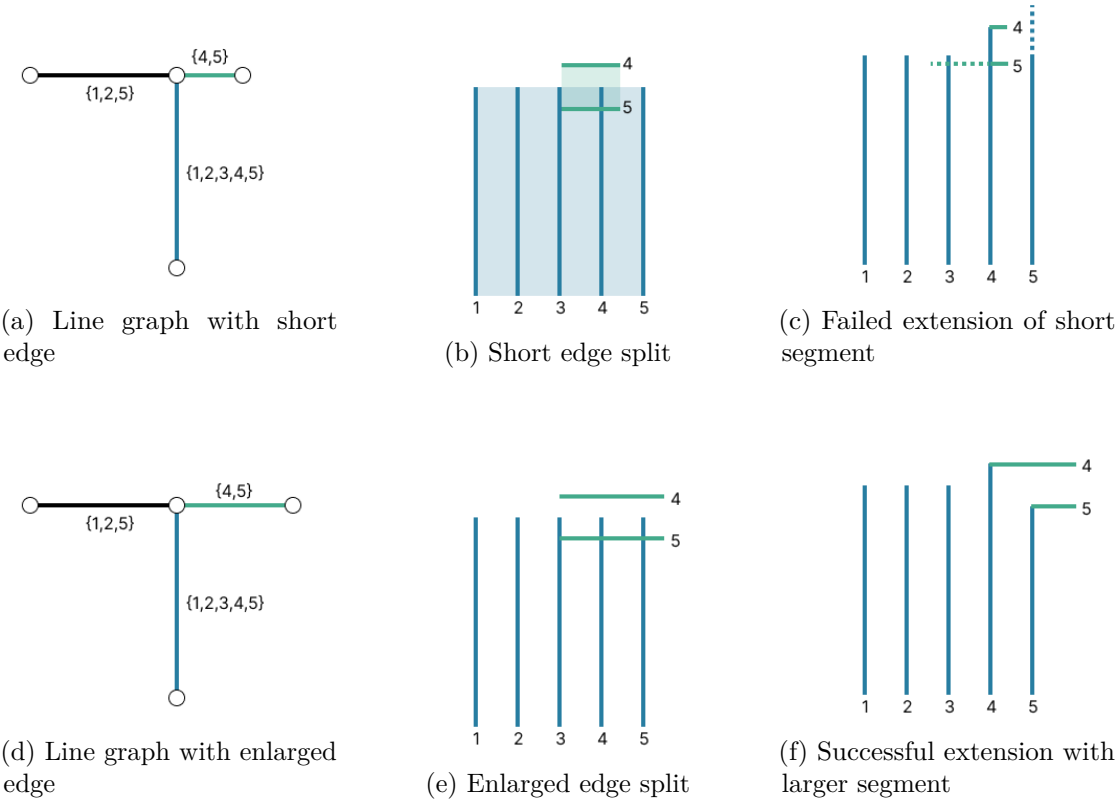
$$\text{For } G = (V, E),$$

$$\text{a path } P_n = (v_0, v_1, v_2, \ldots, v_n)$$

$$\text{such that} \quad (v_i, v_{i+1}) \in E, \quad \forall\, 0 \leq i < n,$$

$$\text{and} \quad \deg(v_i) = 2, \quad \forall\, 1 \leq i \leq n-1.$$

For each path, we construct a geometry by connecting all points along the path. Next, we remove all edges and nodes, except for the first and last ones. Finally, we create a new edge between the first and last nodes of the original path and store the geometry of the path.



(a) Line graph with short edge

(b) Short edge split

(c) Failed extension of short segment

(d) Line graph with enlarged edge

(e) Enlarged edge split

(f) Successful extension with larger segment

Figure 4.11: Comparison of rendering for short vs. enlarged perpendicular edges.

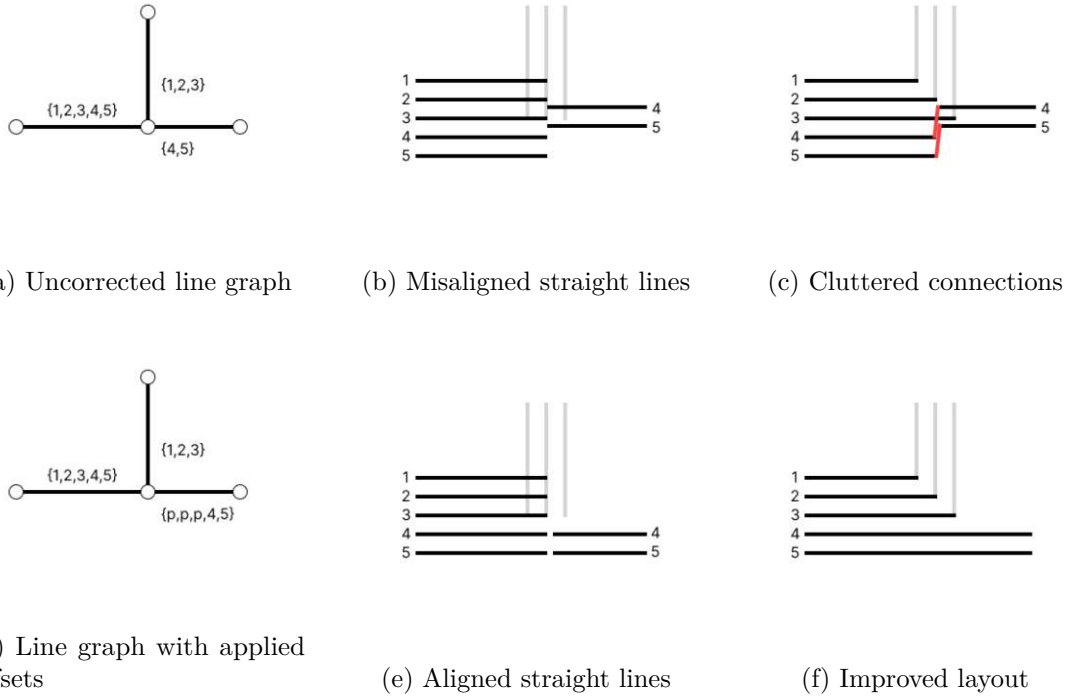
4.5.3 Align straight line edges

The line offsets are calculated based on the track numbers and are always aligned around the center line. If straight lines differ in track count, it is likely that straight-line edges will be offset and form a sharp bend when rendered, see Figure 4.12c.

To handle this issue, we calculate all straight-line connections in the graph and align them based on their line numbers. As shown in Figure 4.12d, the edge with fewer tracks is padded with a placeholder p. The placeholder will not render a line segment but will account for the offset calculation and, therefore, align lines with the same line number, see Figure 4.12e.

For the path calculation, we first find all straight-line paths in our line graph. We iterate over all edges of the graph and try to expand a straight-line path in both directions. We mark each added edge as processed since every edge can only be on one path. We define edges as straight-line connected if the angle formed by them is below 10°.

With the paths calculated, we then proceed with the alignment algorithm. We take the edge with the most tracks as a base and try to align all other edges accordingly. For



(a) Uncorrected line graph (b) Misaligned straight lines (c) Cluttered connections

(d) Line graph with applied offsets (e) Aligned straight lines (f) Improved layout

Figure 4.12: Comparison of line alignment before and after applying offset corrections

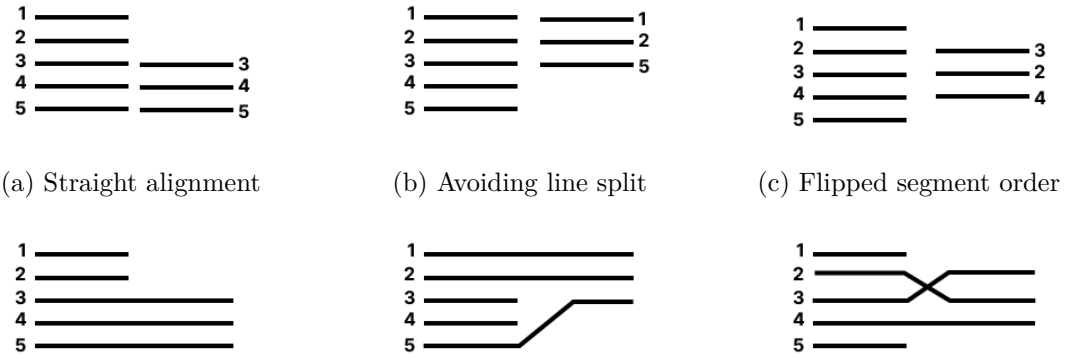
each consecutive segment, we try all constellations and pick the one with the fewest misalignments, which means segments with the same identifier are not on the same track.

We intentionally chose not to split lines for visual clarity. For example, in Figure 4.13b and Figure 4.13e, segment 5 could be aligned as well. However, this would require detaching it from the line bundle, which we avoid. Moreover, such a split could propagate along the entire straight path. In these situations, a short diagonal segment is visually preferable to introducing a disruptive split.

In cases like those shown in Figure 4.13c and Figure 4.13f, the lines are flipped in their segment order. Since perfect alignment isn't possible in this configuration, the algorithm aligns each line to its locally flipped counterpart, as both lines are one-off in this configuration.

4.5.4 Reconstruct Paths

To reconstruct the drawing from the G_{line}, we have to restore the original path of each line. After preparing the graph with cleanup and reducing its complexity by degree-2 node contractions, we can reconstruct paths for each original input line. We first map the original start and endpoints of each input line to the line graph. With this done, we reconstruct the paths by finding the shortest paths between the start and endpoints,



(a) Straight alignment (b) Avoiding line split (c) Flipped segment order

(d) Straight alignment (rendered) (e) Avoiding line split (rendered) (f) Flipped segment order (rendered)

Figure 4.13: Comparison of different alignment constellations.

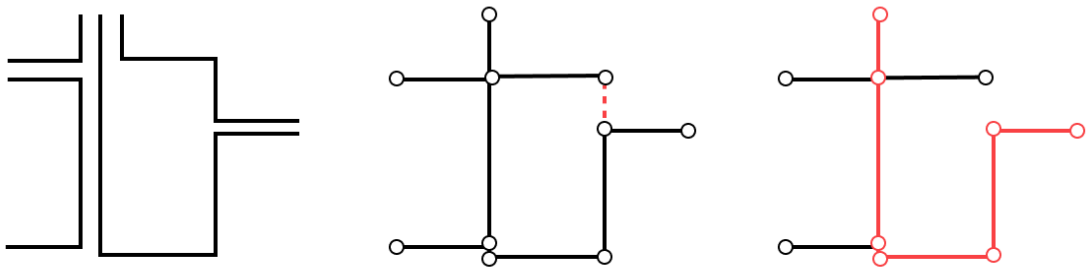
and we use a standard Dijkstra algorithm for the shortest path calculations. This step compensates for potential mislabeling during line graph construction.

Disclaimer on the shortest path calculation

The calculation of shortest paths between start and end points is not guaranteed to yield the correct path. However, in our case, it produced correct paths in most instances. Nevertheless, this is an experimental implementation, and a more robust method must be developed to ensure correct path calculation.

In most cases, it's straightforward to find the original route through the graph, because in most cases it's also the shortest path between start and end nodes. However, there are cases where a simple shortest path does not resemble the original route of the input line. Another issue can arise when the line graph does not capture the connections of the input network correctly. Consider the example in Figure 4.14, the original route cannot be reconstructed, because the path is missing in the line graph.

A modified Dijkstra algorithm can be used to mitigate these issues. The metadata from the line graph can be leveraged to adhere to the original routes. A weight function may be employed that favors paths where the *featureId* of the original line is present. Additionally, an abort threshold should be defined to avoid calculating ambiguous paths that are too far from the original locations of the paths. In such cases, path reconstruction may not be possible. Finally, the running times can be reduced by introducing a heuristic, such as the A* algorithm, which helps to select better candidate nodes during pathfinding.

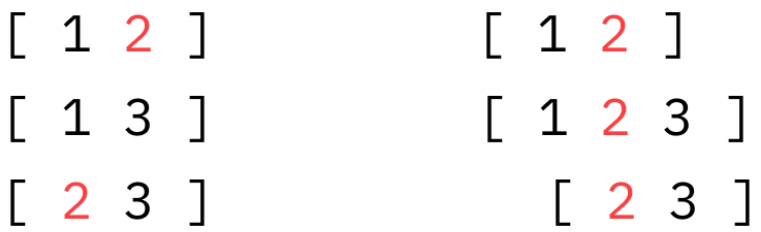


(a) Original (b) Ambigous line graph (c) Invalid path finding

Figure 4.14: Wrong path finding, with ambiguous line graph

4.5.5 Deriving the Order of Missing Lines

Since we construct paths using a shortest path algorithm, the resulting path may pass through edges that originally did not contain the line's **featureId**. As a result, we are unable to determine the correct rendering order of the line for the next step. To address this, we introduce a step to derive the line order based on adjacent edges. Consider the simple example from Figure 4.15. To restore the line order for line 2, we first build a matrix where rows represent the edges of a path, and the columns represent the tracks per segment. Thus, in Figure 4.15a, we have a path covering three segments, and we try to restore the path for line 2. Since line 2 is missing in the second row, we insert it while preserving the order of the first and third rows. In Figure 4.15b, we see that the order remains intact after the insertion. For the calculation of the insert position, we try each configuration and pick the one where the fewest imposed order constraints from other segments are violated.



(a) Route with missing order information (b) Corrected route order information

Figure 4.15: Insertion of missing order information

4.5.6 Offsetting and building the Linestrings

The final Linestring is constructed by iteratively appending offset segments. We first offset the edge geometry by the track width parameter t. We use the offset function of the shapely library to calculate the resulting Linestring.

Since the consecutive segments will overlap, they need to be connected. We use a similar approach as described by Sadahiro, Tanabe, Pierre, and Fujii [33]:

- If the two segments intersect, cut the residual line at the intersection point.
- If the segments do not intersect, attempt to calculate an intersection point by interpolating the segments.
- If the interpolated lines do not intersect, a straight line is added to connect the endpoints. However, this may result in a cluttered path representation, and a more effective alternative should be considered.

4.6 Runtime Analysis

This section concludes the algorithm chapter by providing a runtime analysis. Each step is explained, and an upper bound on its computational complexity is provided. The runtime is expressed in terms of the following variables:

- n - The number of input lines.
- c - The number of component points.
- M - The maximum number of parallel lines per segment.
- P - The maximum number of points per line.
- V - The number of nodes of the initial line graph.
- E - The number of edges of the initial line graph.
- V_2 - The number of nodes in the line graph after degree-2 node contraction.
- E_2 -The number of edges in the line graph after degree-2 node contraction.

The overall asymptotic runtimes are $O(n \log n + nP + n \log c + c \log c)$ for preprocessing, $O(nkP \log P \log n)$ for line graph construction and $O(V + E + n \cdot (V_2 + E_2) \cdot \log V_2 + (n + E_2 + V_2) \cdot PM^2)$ for the render phase. The individual steps contributing to these totals are described below.

4.6.1 Runtime: Preprocessing

First, the lines are sorted using Python's standard `sort` function, which is implemented with Timsort. As a result, the runtime complexity is $O(n \log n)$.

Next, input lines are cleaned, and lines are processed iteratively. Each point in a line is compared with its preceding and succeeding points. Therefore, the runtime for a single line is $O(P)$, where P is the number of points per line. Consequently, the total runtime is $O(nP)$.

Following this, lines are merged based on their component points, where the number of components is denoted as c. We build an R-tree index for point lookup, which is constructed in $O(c \log c)$. Then, we iterate through all n lines and check for matches in the index, where each R-tree lookup has a complexity of $O(\log c)$. Thus, the total complexity is $O(n \log c + c \log c)$.

Summarizing the above steps, the total runtime for the preprocessing phase is:

$$O(n \log n + nP + n \log c + c \log c)$$

4.6.2 Runtime: Line Graph Construction

We iteratively process Lines n, then for each line we do the steps:

- First, we perform a query for nearby edges. The index is built incrementally as we insert lines. The query cost is $O(\log n)$. We define k as the number of nearby edges returned from the query.
- Next, we proceed with line matching. For this, we initialize a priority queue and search for the first matching segment for each edge, which has a complexity of $O(kP \log k)$. The priority queue initially contains k elements. We then process the queue and, for matching intervals, add elements back to the queue. In the worst case, each segment is compared with an edge, leading to an additional complexity of $O(kP \log P)$.
- The number of updates required depends on the number of matches, which is at most P. Updates for G_i are performed in constant time, while index updates have a complexity of $O(\log n)$.

Thus, the total runtime complexity is:

$$O(nkP \log P \log n)$$

4.6.3 Runtime: Render

First, we identify all nodes with a degree not equal to 2. This step depends on the implementation of the *rustworkx* degree function, which we assume runs in constant time per node, resulting in a total complexity of $O(V)$. Next, using the list of potential endpoints, we traverse the graph to find degree-2 paths. Since each node is visited at most twice, the runtime for this step is $O(V + E)$. For the actual removal of these paths, we iterate through all identified degree-2 paths. Since the paths are distinct, the number of traversed nodes is bounded by the total number of nodes, leading to a complexity of $O(V)$. We assume that both edge and node removal operations run in constant time. Thus, the total runtime complexity is $O(V + E)$.

Nearby Node elimination is implemented with DB-Scan clustering, therefore $O(N^2)$ in runtime.

The pathfinding, where we perform a Dijkstra search for each input line, has a complexity of $O(n \cdot (V_2 + E_2) \cdot \log(V_2))$, since we use the *rustworkx* Dijkstra implementation.

For the next step, we insert the missing line identifiers in the correct order. We iterate through all paths, corresponding to the number of input lines n. Then, we iterate over the paths to detect conflicts, which are bounded by the maximum path length P. In our case, P is relatively small, since powerlines are typically local. The next step in the loop is to resolve the detected conflicts, which has a complexity of $O(PM^2)$, where M is the maximum number of lines per segment. Although the total runtime is $O(nPM^2)$, since both P and M are small in our case, this step is relatively insignificant in terms of the actual runtime.

We first perform a depth-first search (DFS)-like expansion, which has a complexity of $O(E_2 + V_2)$. Then, for each straight-line segment, we perform an alignment, which has a complexity of $O(PM^2)$. Therefore, the total runtime is $O((E_2 + V_2) \cdot PM^2)$. Note that we assume the straight-line path lengths to be shorter than the powerline path lengths, and for simplicity, we use P to represent the maximum path length.

For the final step, offset lines are constructed. We first iterate through all n paths, and then for each path, we iterate over each segment P to perform the geometric operations needed to concatenate the lines, which takes $O(1)$ time per operation. Therefore, the total runtime is $O(nP)$.

To summarize, the total runtime of the render step is:

$$O(V + E + n \cdot (V_2 + E_2) \cdot \log V_2 + nPM^2 + (E_2 + V_2) \cdot PM^2 + nP)$$

which can be simplified to:

$$O(V + E + n \cdot (V_2 + E_2) \cdot \log V_2 + (n + E_2 + V_2) \cdot PM^2)$$

CHAPTER 5

Evaluation

To evaluate the algorithm, we define a set of evaluation metrics along with threshold values to classify instances as either passed or failed. In addition, we assess the overall drawing quality and interpret the resulting metrics. For a more detailed analysis, we select 50 test instances to examine error cases and identify distinct classes of problems objectively. Next, we evaluate the algorithm's performance on real-world data. Finally, we conclude the evaluation by discussing the extent to which each requirement has been fulfilled, based on Section 3.4.

5.1 Data and Parameter Settings

This section provides an overview of the dataset used for evaluation and outlines the key parameters applied in the experiments.

5.1.1 Data

The data used for evaluation originates from a real-world low-voltage network ("Niederspannungsnetz") in Vienna. As this data is subject to confidentiality, only aggregated statistical metrics are presented. Example visualizations are anonymized and do not allow inference of the original locations. Specifically, all geographic coordinates, street names, and identifying labels have been removed.

We begin our analysis with the input data, using the complete real-world dataset *g100*. A summary of this dataset is presented in table 5.1. After the line-merging step, the initial 433k lines are reduced to 311k lines. For the sake of simplicity in the comparison, we use the merged lines ($n = 311k$).

After constructing the line graph, we obtain a graph comprising over one million nodes and edges (refer to table 5.2). The average degree of the graph is below 2, indicating

Name	Max Parallel	Line Count	Bends	Crossings	Length(km)	Area(km^2)
g100	22	433605	3098274	0	12308.9	10109.5

Table 5.1: Statistics of the real-world dataset *g100*.

that it is sparse. Furthermore, the graph consists of 93,205 disconnected components, resulting in an average of approximately 10 nodes and edges per component. The five largest components are of size: 204372, 101529, 29742, 28996, and 21197. This highlights that despite the large number of small components, the main structural areas of the network are still captured within a few large, connected subgraphs.

Name	Nodes	Edges	Average Degree	Connected Components
g100	1195506	1143671	1.9133	93205

Table 5.2: Statistics of the line graph derived from dataset *g100*.

5.1.2 Parameters

The following parameter values are used consistently across all evaluations.

- **Input line spacing:** 0.2 m
- **Generalized line spacing:** Variable (1x–8x of input)
- **Line width:** 5 m
- **Threshold angle:** 5 degree

Input Line spacing

The line spacing is the distance between parallel lines in the input drawing, which is 0.2m in our dataset.

Generalized Line spacing

The line spacing is the distance between

To get a better picture of how the generalized line distance influences the output, we define four scales:

- 1x: 0.2m, which is equal to the original line distance
- 2x: 0.4m
- 4x: 0.8m

- 8x: 1.6m, which provides a drawing readable at a target scale of 1:2000, as defined by requirement **R3**

Line width

The line width is the scan distance used to calculate the lines that should be merged during the construction of the skeleton network.

We use a fixed line width of 5 m for all test runs. This value was derived from the maximum number of parallel tracks observed in our dataset, which was 22. The line width must be sufficient to cover the longest parallel arrangement; otherwise, the corresponding line segments cannot be merged. This represents a limitation of the approach, as the required width must be known in advance: a non-trivial task when working with large datasets. To merge the 22 parallel lines with a spacing of 0.2 m each, we need to set the line width to $22 \times 0.2\,\text{m} = 4.4\,\text{m}$. To account for the worst case, when the two outermost lines are merged, a width of 5 m allows for a reasonable error margin.

Threshold angle

The threshold angle is used during preprocessing to detect and remove nearly collinear points.

5.2 Evaluation Metrics

The first metric is the pairwise Fréchet distance, which compares each input line with its corresponding generalized output line. The Fréchet distance provides a reliable measure of geometric similarity, capturing the overall shape and trajectory of the lines.

The second metric is the number of crossings per line, measured before and after generalization. This metric is designed to detect misalignments between multiple lines and to assess the topological consistency among them: something the Fréchet distance does not account for.

The third metric is the count of acute angles, where we identify angles below 30 degrees. This captures cases in which the Fréchet distance may indicate a good match due to spatial proximity, even though the generalized line introduces several sharp bends that degrade visual or structural quality.

For all three metrics, the optimal value is 0, indicating that the generalized output matches the input both geometrically and topologically, without introducing undesired artifacts.

5.2.1 Fréchet Distance

To evaluate the test cases, we employ the Fréchet distance to compare the generalized results with the original data. Specifically, we measure the Fréchet distance for each

pair of original and corresponding generalized lines. This metric captures the geometric deviation between the two line representations.

The Fréchet distance is influenced by the generalization factor: larger factors result in greater displacement from the original positions. Moreover, the number of parallel lines impacts the displacement as well. Outer lines tend to undergo more significant shifts compared to inner lines, and this effect intensifies as the number of parallel lines increases during generalization.

For the evaluation, we use the continuous Fréchet distance, as it offers higher accuracy compared to the discrete approximation, despite its greater computational cost [20].

Calculation of Threshold

To evaluate the Fréchet distance meaningfully, we need a threshold that defines what counts as an acceptable deviation. Since we evaluate lines individually, without access to structural or topological context, we require a reference value to judge whether the distance between an original and a generalized line is reasonable.

The threshold represents an **expected average deviation**: a distance below which a generalized line is still considered a reasonable match to its original counterpart, while still penalizing significantly distorted lines. Since outer lines are translated farther than inner ones in a bundle, the threshold accounts for this variation by considering two key parameters:

- The number of parallel lines N, as it determines how far from the original line the generalized line is placed. As shown in Figure 5.1, the individual line offsets are symmetric and grow with the total number of lines.
- The generalizion line distance d, lines are translated by multiples of d.

To account for those parameters, we start with the given expression, with a maximum number of parallel lines N and the distance between lines in the generalized drawing. We calculate the average deviation of the line given the two parameters, and use the symmetric properties of a line bundle.

$$\begin{aligned}\frac{1}{N} \cdot 2 \sum_{i=1}^{N/2} (i \cdot d) = \frac{2d}{N} \sum_{i=1}^{N/2} i = \frac{2d}{N} \cdot \frac{(N/2)(N/2+1)}{2} \\ = \frac{2d}{N} \cdot \frac{N^2/4 + N/2}{2} = \frac{2d}{N} \cdot \frac{N^2+2N}{8} \\ = \frac{2d(N^2+2N)}{8N} = \frac{d(N^2+2N)}{4N} = \frac{dN(N+2)}{4N} \\ = \frac{d(N+2)}{4}\end{aligned}$$

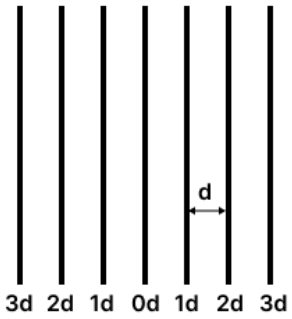


Figure 5.1: Expected offsets of parallel lines, with generalized line distance d

5.2.2 Line Crossings

The second metric evaluates line crossings by counting how often each line intersects with any other line. Since each crossing involves two lines, the metric is symmetric; for example, when lines A and B are crossing, we will increase the crossing count for A and B. We do not track which specific lines intersect, but instead record the total number of crossings per line. This simplified approach avoids the computational overhead of tracking all n^2 potential crossing pairs, which would be infeasible given the dataset of over 350,000 input lines. Self-intersections are not included in the crossings metric but are captured as a separate metric. Because self-intersections are non-trivial to calculate for a single geometry, we only record whether any self-intersections exist, not their count.

To assess the quality of the output, we apply a binary criterion. For each input line, we compare the number of crossings in the input with the number in the output. If the count remains unchanged, the line is considered to have passed the crossing metric.

In Figure 5.2, we show the distribution of crossing counts for the original data. Most lines do not exhibit any crossings, while a smaller fraction falls within the range of up to five crossings. Only a few outliers have significantly higher crossing counts.

5.2.3 Acute Angles

Acute angles are considered difficult to interpret in planar drawings, as discussed in the literature Purchase [30], Fleischer and Hirsch [22], and Ware, Purchase, Colpoys, and McGill [39].

We define acute angles as those measuring 30 degrees or less. In the original dataset, we identified 1,470 lines that form such angles, which constitute approximately 0.5% of all lines. This suggests that sharp bends are relatively rare in the original dataset.

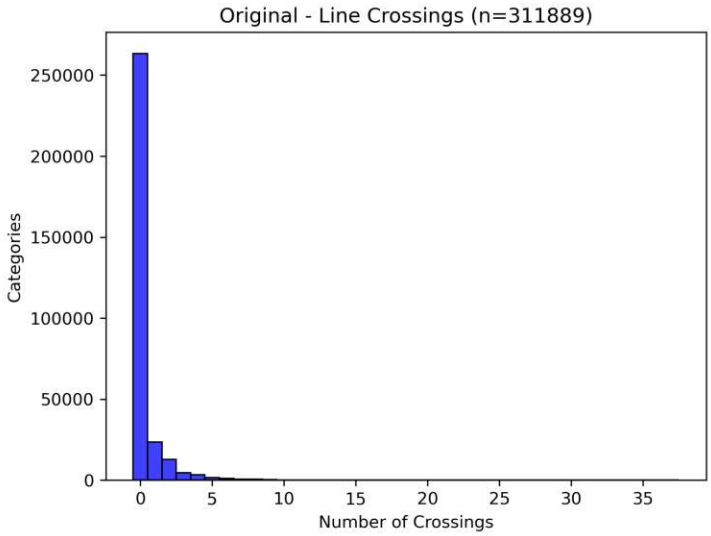
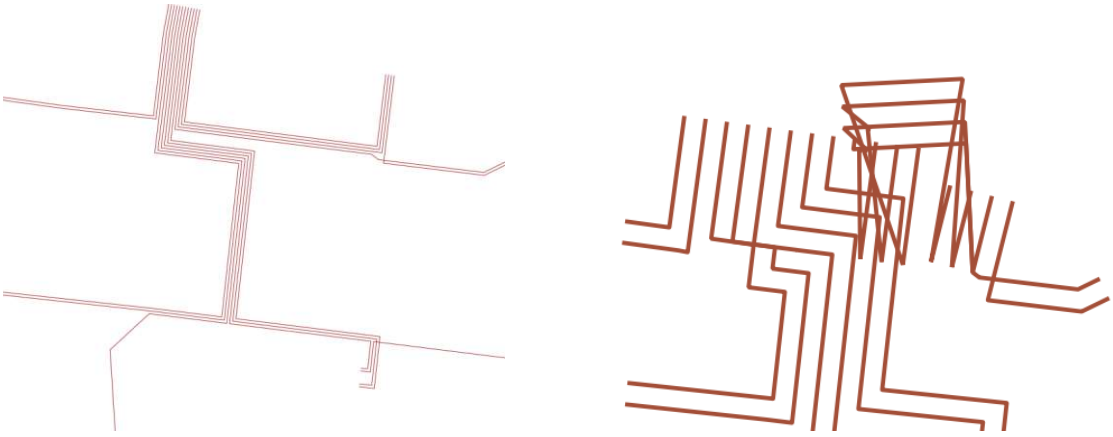


Figure 5.2: Number of crossings per line in the original data

We consider it a failure when an individual line that originally contained no acute angles develops one or more in the output. We do not count the exact number of acute bends per line, only whether such a change occurred.

The rationale for using line bends as a metric is based on the fact that the original drawing contains few sharp bends. In most cases, the introduction of such bends indicates errors in the generalization process. In the example shown in Figure 5.3, we observe the erroneous artifacts introduced during the 8x generalization, where some lines exhibit very sharp bends.



(a) Test case 1 (original) (b) Test case 1 (generalized, 8x) Acute bends

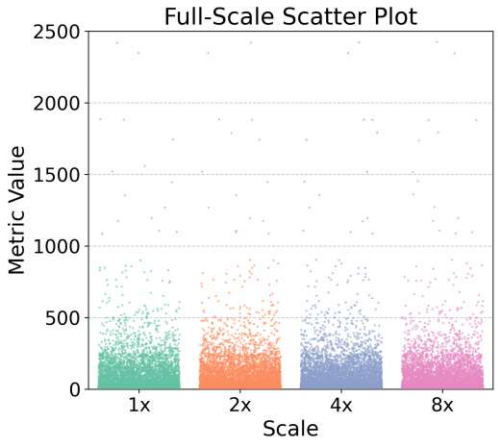
Figure 5.3: Example of acute bends

5.3 Global Drawing Quality

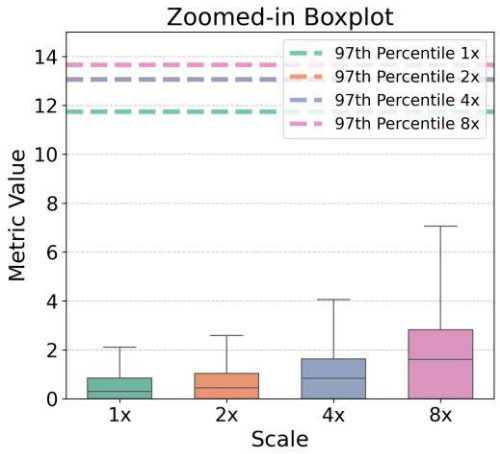
We assess the global drawing quality using our defined metrics. Initially, we evaluate each metric independently and then provide a comprehensive overview to classify the overall outcome.

5.3.1 Fréchet Distance Analysis

As shown in Figure 5.4a, the measured Fréchet distances range from 0 m to 2,500 m, with the majority of values concentrated below 500 m. Distances exceeding 500 m are classified as extreme outliers; a detailed analysis of these cases is provided in Subsection 5.4.4.



(a) Fréchet distances visualized with strip plot



(b) Zoomed view Fréchet Boxplots

Figure 5.4: Fréchet distance Plots

To better visualize the distribution at the lower end, a zoomed-in boxplot is provided in Figure 5.4b, with outliers excluded for clarity (as they are already depicted in the scatter plot). Approximately 97% of the data lies below 14 m at the 8x scale and below 12 m at the 1x scale. The remaining 3%, corresponding to approximately 9,330 lines, are classified as outliers.

The upper whisker extends from around 2 meters at the 1x scale to approximately 7 meters at the 8x scale. While such deviations may be acceptable in dense networks with many parallel lines, they are problematic in networks with low parallelism, where deviations should ideally be close to zero.

The third quartile remains below 1 meter at 1x and below 5 meters at 8x, suggesting that most of the data is reconstructed with high precision. However, it must be noted that a substantial proportion of the network consists of non-parallel line segments, for which minimal deviations are expected, effectively lowering the overall distribution bounds.

In conclusion, while the majority of data points fall within an acceptable deviation range, the presence of heavy outliers is concerning and warrants removal. Additionally, the subset of lines within the upper whisker range represents borderline cases that may require further refinement. The overall distribution is skewed toward lower distances, partly due to the prevalence of non-parallel line segments in the dataset.

5.3.2 Crossings Count

The crossings metric fails in approximately 27.3% to 32.2% of cases, depending on the scale, with performance worsening as the scale increases. In Figure 5.5, data points with a zero difference (i.e., no change in crossings) were excluded to better emphasize the deviations. Additionally, a small number of extreme outliers with differences above 50 were omitted from the plot: 402 values for 1x, 373 values for 2x, 331 values for 4x, and 288 values for 8x.

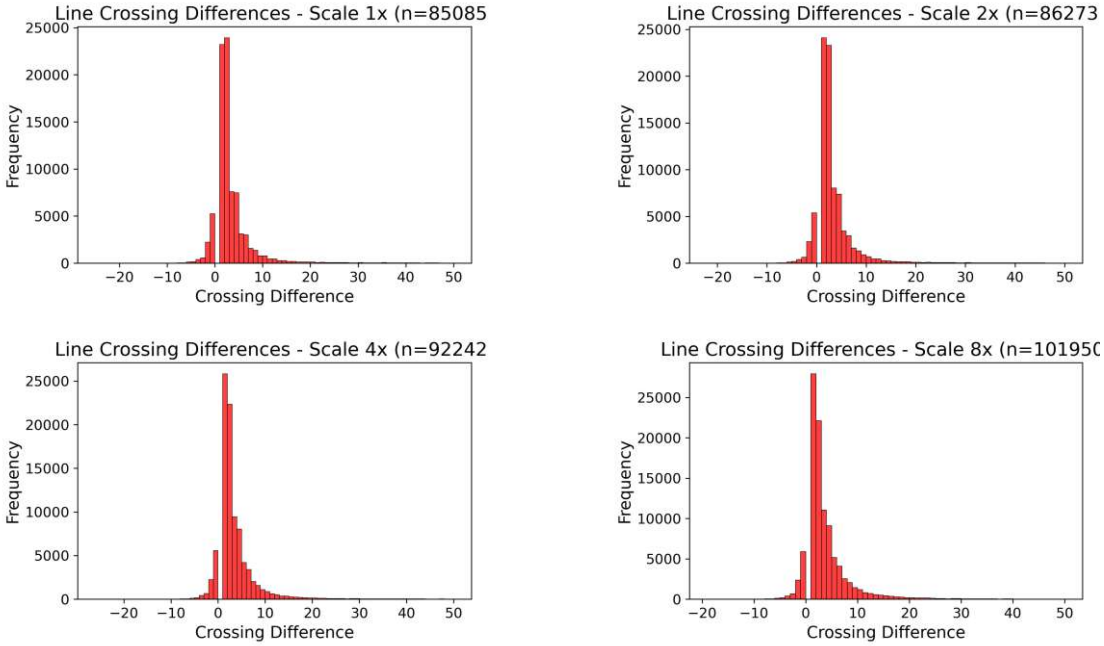


Figure 5.5: Differences in Line Crossings Between Input and Output

Most observed differences fall within the range of 1 to 10, while only a few are negative, indicating that the number of crossings in the output typically increases compared to the input.

As shown in Figure 5.6, the number of self-intersections increases with scale. Starting from 1085 self-intersections in the original data, the 8× scale contains 7451. Although these numbers are low compared to the total number of lines, newly introduced self-intersections should be avoided. The trend shows an increased number of self-intersecting lines as the scale gets smaller.

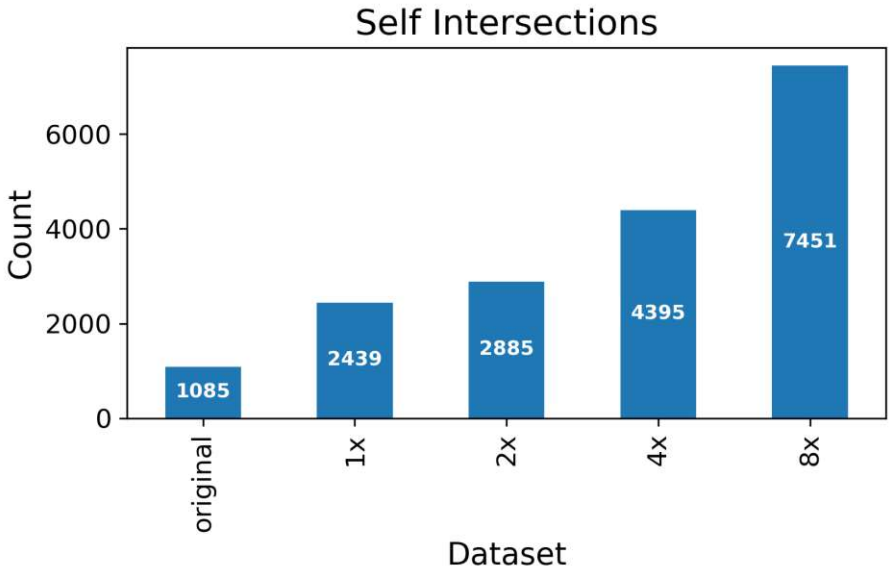


Figure 5.6: Self-intersections per scale

Recall the symmetric property of the crossings metric. In Figure 5.7, we illustrate the strictness of this metric. Although the drawing appears mostly correct, aside from two clear artifacts, only a single line passes the crossing check. The upper artifact intersects with five other lines, and the second artifact introduces several additional crossings. As a result, only one line in the example remains crossing-free.

Considering this, the crossing metric is a sharp test, but considering the heavy influence of crossings of unrelated lines on the perceived drawing quality, it is viable to punish these cases.

(a) Test Case 34, original

(b) Test Case 34, generalized 8x

Figure 5.7: Test Case 34, Display of Line Crossings Metric

5.3.3 Classification

For a summarized evaluation, each output line is assigned to a category based on the results of the evaluation metrics. A line is labeled *ok* if it passes all checks, and *missing* if it is not included in the output set. If exactly one metric fails, the line is labeled accordingly: *Fréchet distance failed*, *Crossing count failed*, or *Acute bends failed*. If more than one metric fails for a line, it is labeled *Multiple failed*.

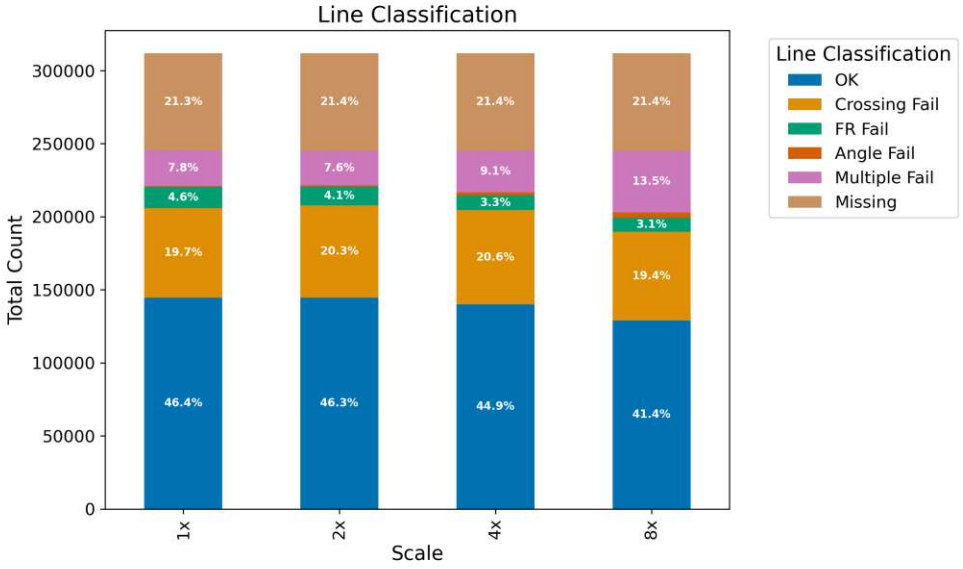


Figure 5.8: Line Result Classification

The measurements are presented in Figure 5.8, where each bar represents the total number of lines at each scale. The total number of input lines is 311,000. We observe that the proportion of lines for which all checks pass decreases from 46.4% to 41.4% as the scale increases, with the largest drop occurring at the 8x scale.

The proportion of lines failing the crossing metric remains relatively constant at around 20%. However, there is a noticeable increase in the proportion of lines failing multiple metrics, suggesting that the total number of crossing failures also rises with larger scales. The acute angle metric affects only a small subset of lines but exhibits a slight upward trend as the scale increases. This observation suggests that acute angles rarely occur in isolation and are more likely to appear in conjunction with crossing errors or with excessive Fréchet distances.

A particularly concerning observation is the significant portion of lines that are missing from the output set, consistently around 21.4%. The cause of this issue is analyzed in more detail in Subsection 5.4.4. Overall, the trend indicates a high failure rate, with the large number of missing lines in the output being especially problematic.

5.3.4 Discussion of Results

In the global evaluation, the most critical issue identified was the high number of missing lines in the output. This is primarily caused by incomplete connectivity in the constructed line graph, where expected topological links between segments are absent. This significantly impacts the completeness and structural integrity of the output Subsection 5.4.4.

The requirement for improved readability (**R2**) is only partially fulfilled. In certain regions, the generalization improves readability compared to the original drawing. However, fewer than half of the lines meet the defined quality thresholds, indicating a clear need for refinement in the rendering process.

5.4 Local Test Cases

Due to the large size of the original dataset, precise measurements can only be obtained approximately. Furthermore, a substantial portion of the data consists of simple linear connections, which tend to distort the evaluation of the truly challenging areas, particularly regions with parallel lines and complex junctions.

5.4.1 Test Case Selection

We manually selected 50 test case instances (table 5.3) to capture a broad range of challenges and ensure diversity across scenarios. Each test case includes at least some parallel lines, typically ranging from 3 to 20. Various types of junctions are represented, with some cases containing line crossings and others not. Additionally, certain cases intentionally include features that are known to be error-prone.

However, specific geometric structures, such as clusters of poles forming distinct star-like patterns, are excluded, as they differ substantially from the typical geometry of the network and would distort the evaluation focus.

Name	Max Parallel	Line Count	Bends	Total Length	Crossings	Area
tc0	15	16	89	400.74	9	110.39
tc1	12	16	92	429.03	3	358.40
tc2	9	9	46	113.44	7	72.87
tc3	12	13	79	541.91	27	559.94
tc4	16	18	106	368.17	7	124.51
tc5	9	12	71	248.13	4	173.21
tc6	7	12	73	148.65	6	1.45
tc7	6	9	56	138.33	14	57.70
tc8	6	12	37	123.62	3	48.11
tc9	9	11	51	225.32	2	37.13

Continued on next page

Name	Max Parallel	Line Count	Bends	Total Length	Crossings	Area
tc10	13	19	143	405.42	0	133.06
tc11	12	25	102	332.83	4	41.14
tc12	25	27	112	945.67	0	361.79
tc13	9	12	62	291.54	0	268.10
tc14	10	13	79	375.42	0	309.71
tc15	20	27	218	607.62	10	92.25
tc16	7	14	54	145.31	0	16.90
tc17	9	12	77	383.30	0	6.35
tc18	8	10	142	683.87	4	988.91
tc19	7	11	39	207.94	6	92.50
tc20	11	11	73	233.51	2	90.66
tc21	18	27	136	1189.05	39	4.93
tc22	8	18	78	298.74	11	68.30
tc23	8	14	60	204.36	3	0.65
tc24	10	24	81	441.32	6	11.35
tc25	9	9	35	134.45	12	49.40
tc26	8	8	60	174.48	0	122.55
tc27	9	10	68	253.16	11	171.48
tc28	6	7	59	286.29	4	170.80
tc29	8	16	112	375.71	0	378.57
tc30	9	10	49	130.83	0	115.59
tc31	10	14	61	276.15	2	89.80
tc32	7	12	42	191.05	0	138.47
tc33	8	11	135	330.30	0	338.61
tc34	10	16	71	276.76	0	94.05
tc35	6	15	175	507.12	17	446.01
tc36	9	14	96	555.07	1	590.67
tc37	8	16	56	166.28	0	37.23
tc38	8	14	65	368.41	0	1.34
tc39	18	23	218	994.03	23	297.94
tc40	5	12	80	180.51	9	29.10
tc41	5	15	74	237.98	5	161.09
tc42	14	21	143	686.38	5	739.04
tc43	10	11	53	168.34	0	122.90
tc44	4	8	38	152.33	0	81.04
tc45	11	27	123	391.16	9	240.12
tc46	6	9	33	100.43	0	41.43
tc47	6	9	201	619.48	0	2106.91
tc48	7	14	71	258.04	2	96.03
tc49	10	14	123	379.26	20	367.68

Continued on next page

Name	Max Parallel	Line Count	Bends	Total Length	Crossings	Area
tc50	12	11	68	459.49	0	815.31

Table 5.3: Local Test Case instances

5.4.2 Experiment Setup

To evaluate each test case, we generate four output drawings: **1x**, which corresponds to the original input scale, as well as **2x**, **4x**, and **8x**. This setup provides a comprehensive overview and facilitates the detection of errors or limitations that may arise with increasing scale.

Finally, we compute the evaluation metrics for each test case and configuration. These values are then compared against predefined thresholds to determine whether a test case passes or fails. In particular, we compare the average Fréchet distance against its corresponding threshold to assess geometric similarity.

5.4.3 Results

The results are summarized in table 5.4. For each test case, we report the maximum number of parallel lines, as this value is used to compute the threshold values. In the middle section of the table, the calculated thresholds for each scale are presented. On the right, we show the measured average Fréchet distance for each test case and scale. Measurements are highlighted in green if they pass (i.e., the distance is below the threshold) and in red if they fail.

A clear trend emerges: lower scales consistently perform better than higher scales. This pattern aligns with both the global results and our subjective assessment of the drawings. Notably, some test cases fail across all scales, indicating that when the initial line graph construction is flawed, no subsequent scaling can produce an acceptable result.

Name	#Parallel	Fréchet Threshold				Fréchet Result			
		1x	2x	4x	8x	1x	2x	4x	8x
tc0	15	3.62	4.28	5.35	6.91	**0.94**	**1.43**	**3.41**	7.41
tc1	12	3.04	3.58	4.46	5.74	**1.03**	**1.24**	**2.46**	5.99
tc2	9	2.45	2.88	3.57	4.58	**0.75**	**1.69**	**2.9**	5.41
tc3	12	3.04	3.58	4.46	5.74	**2.0**	**2.2**	**3.24**	6.64
tc4	16	3.82	4.51	5.65	7.3	**2.7**	**2.6**	**4.87**	9.76
tc5	9	2.45	2.88	3.57	4.58	**0.75**	**1.31**	**2.7**	5.9
tc6	7	2.06	2.41	2.97	3.8	2.18	2.52	**2.71**	4.4
tc7	6	1.86	2.17	2.68	3.41	**0.74**	**0.84**	**1.49**	**3.08**
tc8	6	1.86	2.17	2.68	3.41	**1.07**	**1.46**	**2.14**	4.03
tc9	9	2.45	2.88	3.57	4.58	**0.54**	**0.73**	**1.69**	**3.77**

tc10	13	3.23	3.81	4.75	6.13	3.63	3.83	4.79	8.04
tc11	12	3.04	3.58	4.46	5.74	**0.55**	**0.76**	**1.76**	**3.98**
tc12	25	5.58	6.62	8.32	10.8	**1.61**	**2.5**	**5.44**	11.59
tc13	9	2.45	2.88	3.57	4.58	**0.94**	**1.1**	**2.16**	**4.51**
tc14	10	2.65	3.11	3.86	4.97	**1.0**	**1.23**	**2.25**	**4.44**
tc15	20	4.6	5.45	6.83	8.86	**1.71**	**2.3**	**4.22**	**7.37**
tc16	7	2.06	2.41	2.97	3.8	9.82	9.92	9.94	10.53
tc17	9	2.45	2.88	3.57	4.58	7.78	5.93	4.51	9.08
tc18	8	2.26	2.64	3.27	4.19	**0.69**	**0.81**	**1.75**	**3.78**
tc19	7	2.06	2.41	2.97	3.8	**0.88**	**0.92**	**1.55**	**3.2**
tc20	11	2.84	3.34	4.16	5.36	**2.28**	**2.63**	**3.84**	5.78
tc21	18	4.21	4.98	6.24	8.08	**1.65**	**2.58**	**5.75**	13.81
tc22	8	2.26	2.64	3.27	4.19	5.68	6.52	6.68	7.09
tc23	8	2.26	2.64	3.27	4.19	**0.88**	**1.11**	**1.94**	**3.87**
tc24	10	2.65	3.11	3.86	4.97	**1.46**	**1.58**	**2.36**	**4.44**
tc25	9	2.45	2.88	3.57	4.58	5.63	10.77	10.85	11.36
tc26	8	2.26	2.64	3.27	4.19	**0.9**	**1.04**	**1.87**	4.27
tc27	9	2.45	2.88	3.57	4.58	**1.57**	**1.54**	**2.1**	**3.78**
tc28	6	1.86	2.17	2.68	3.41	**1.76**	**1.81**	**2.28**	3.46
tc29	8	2.26	2.64	3.27	4.19	**0.68**	**0.85**	**1.68**	**3.74**
tc30	9	2.45	2.88	3.57	4.58	**0.9**	**1.03**	**1.79**	**3.58**
tc31	10	2.65	3.11	3.86	4.97	**0.52**	**0.72**	**1.68**	**3.28**
tc32	7	2.06	2.41	2.97	3.8	**0.95**	**1.01**	**1.68**	**3.33**
tc33	8	2.26	2.64	3.27	4.19	**1.15**	**1.29**	5.41	7.02
tc34	10	2.65	3.11	3.86	4.97	**1.2**	**1.56**	**2.64**	5.44
tc35	6	1.86	2.17	2.68	3.41	3.2	4.2	6.43	7.31
tc36	9	2.45	2.88	3.57	4.58	4.53	**2.6**	3.87	7.05
tc37	8	2.26	2.64	3.27	4.19	**0.51**	**0.67**	**1.47**	**3.17**
tc38	8	2.26	2.64	3.27	4.19	**0.61**	**0.84**	**1.74**	4.48
tc39	18	4.21	4.98	6.24	8.08	**2.5**	**2.85**	**4.53**	**7.8**
tc40	5	1.67	1.94	2.38	3.02	**1.33**	**1.45**	**1.79**	**2.76**
tc41	5	1.67	1.94	2.38	3.02	**0.84**	**1.15**	**1.82**	3.26
tc42	14	3.43	4.05	5.05	6.52	4.22	4.54	5.58	7.75
tc43	10	2.65	3.11	3.86	4.97	**1.57**	**2.19**	**3.65**	5.93
tc44	4	1.47	1.7	2.08	2.63	5.45	5.63	6.05	5.38
tc45	11	2.84	3.34	4.16	5.36	**1.65**	**1.69**	**2.81**	**5.23**
tc46	6	1.86	2.17	2.68	3.41	**1.59**	**1.69**	**2.59**	4.34
tc47	6	1.86	2.17	2.68	3.41	**0.86**	**1.07**	**2.01**	4.24
tc48	7	2.06	2.41	2.97	3.8	**0.96**	**1.14**	**1.73**	**3.4**
tc49	10	2.65	3.11	3.86	4.97	**2.28**	**1.78**	**2.48**	5.02
tc50	12	3.04	3.58	4.46	5.74	**0.6**	**0.79**	**1.87**	**4.18**

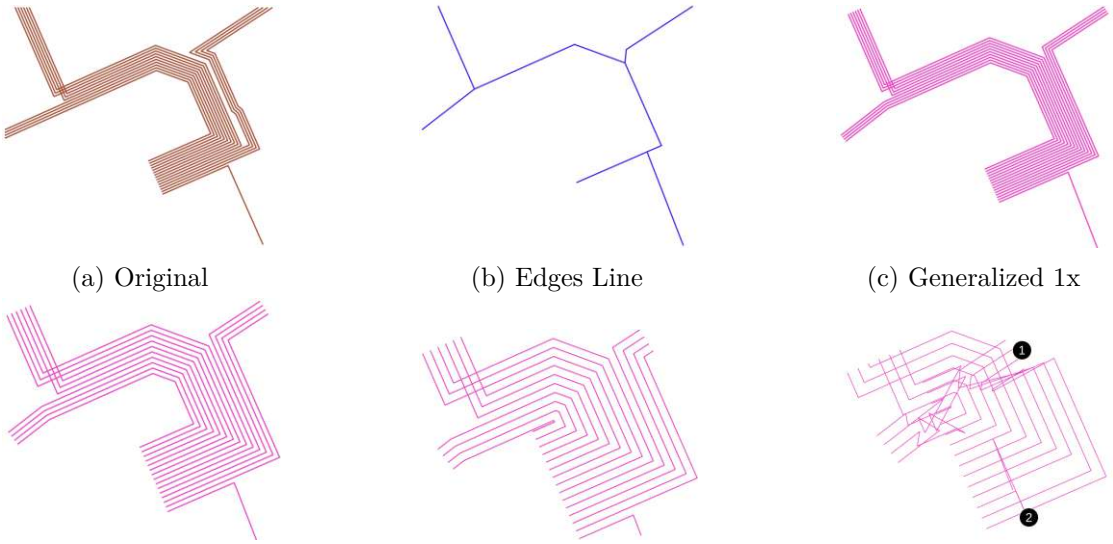
Table 5.4: Local Test Case Results (Passed test cases are highlighted)

5.4.4 Error Classes

In this section, we use the test cases as a basis to identify underlying issues in the algorithm and classify them into distinct error classes.

Displacement Issues

During generalization, particularly for smaller output scales, features that originally did not intersect may be displaced such that they come into proximity or even intersect. These issues become more prevalent at higher output scales, as increased line spacing effectively compresses the surrounding geometry. Although we introduced a perpendicular line offset in Subsection 4.5.1 to mitigate such cases, this heuristic resolves only a limited number of instances.



(a) Original (b) Edges Line (c) Generalized 1x

(d) Generalized 2x (e) Generalized 4x (f) Generalized 8x

Figure 5.9: Test Case 0.

In our test cases, this displacement issue appears prominently in cases `tc0`, `tc12`, `tc13`, `tc34`, and `tc38`. As illustrated in Figure 5.9, several problems arise from the proximity of transformed line segments. Initially, the line graph is correctly constructed from the input data. For scales **1x**, **2x**, and **4x**, the rendering functions as expected: the lines maintain their correct order without introducing new crossings or unwanted artifacts.

However, observe the gap between groups of lines on the top and bottom in the figure. As the scale increases, this gap naturally decreases. Because the current system lacks a mechanism to push lines apart, the **8x** drawing suffers from clutter: lines from both groups are interlaced, making a crossing-free rendering impossible.

Additional problems are visible at locations (1) and (2). These arise due to insufficient segment length at the line ends. Our rendering algorithm fails to generate valid connec-

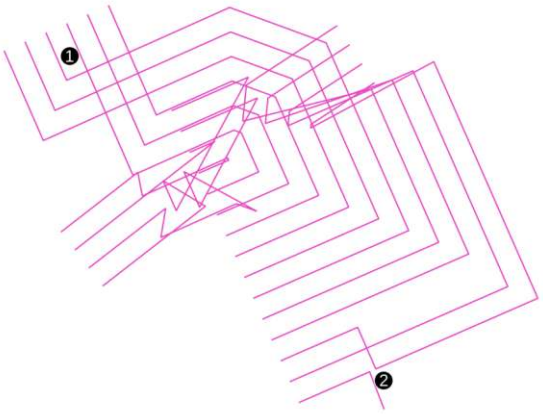


Figure 5.10: Test Case 0, generalized 8x with perpendicular minimum length constraint

tions in such cases, triggering the fallback method. This method connects endpoints with straight lines, which results in visual clutter. By applying the perpendicular edge length constraint introduced earlier, these problems can be mitigated. In Figure 5.10, we show how this constraint successfully resolves the issue by increasing edge lengths at (1) and (2). However, this solution works in this isolated test case, since the lines terminate at that point. However, it would not be viable for the global dataset, where lines likely continue beyond the local context. This underscores the need for a more robust global displacement strategy that adjusts line positions based on proximity while preserving topological and geometric integrity.

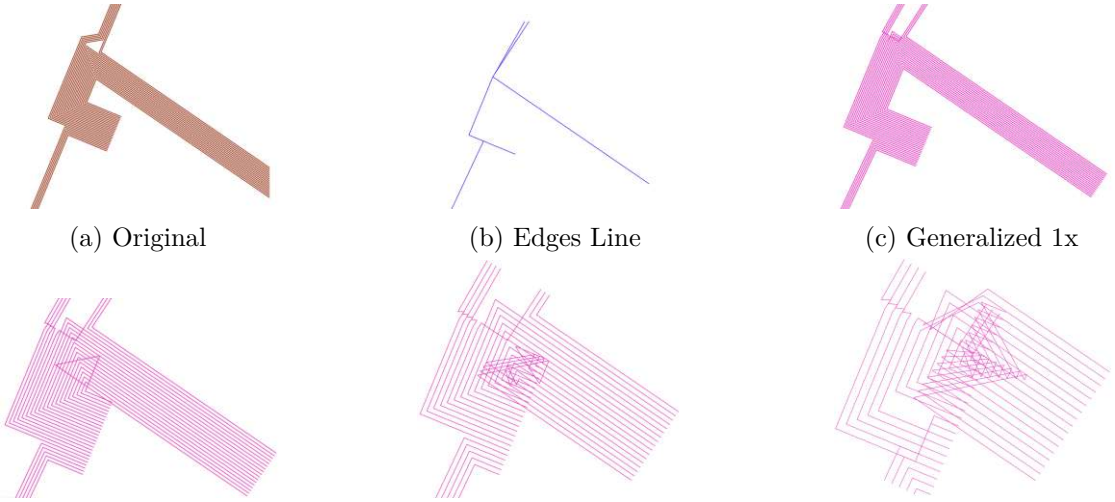
A similar issue is evident in `tc12`, where a greater number of parallel lines and a sharper angle cause overlap to occur as early as the **2x** scale. In this case, a single line overlaps another, creating a triangular artifact. Whether such self-intersections can always safely be removed and how to handle more complex configurations remains an open research question.

Line Graph Construction – Geometric Issues

The next category of errors arises from issues in the line graph construction, specifically when the geometry of the input lines is not accurately captured. As a result, the derived output lines become ambiguous, leading to distorted or misleading representations.

An illustrative example is provided in Figure 5.12, which depicts an otherwise simple test case complicated by the presence of a geometrically uncommon sink line. This results in an ambiguous connection within the line graph, ultimately distorting the output geometry.

The core of the issue lies in the fixed line detection width of 5 meters. While this width is necessary for handling larger test cases with many parallel lines, or for the global dataset, it proves too coarse in this instance. As a consequence, fine-grained geometric details are

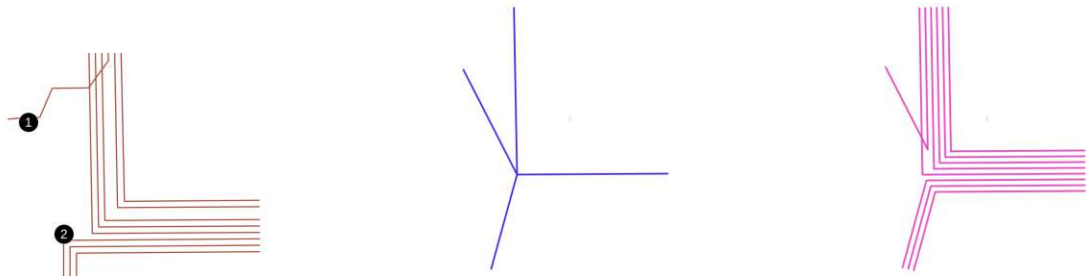


(a) Original (b) Edges Line (c) Generalized 1x

(d) Generalized 2x (e) Generalized 4x (f) Generalized 8x

Figure 5.11: Test Case 12.

lost. In particular, the line marked (1) is incorrectly matched with the line marked (2), simply because they lie within the 5-meter threshold. Although these lines are unrelated, the algorithm falsely groups them, introducing an artifact into the final rendering.



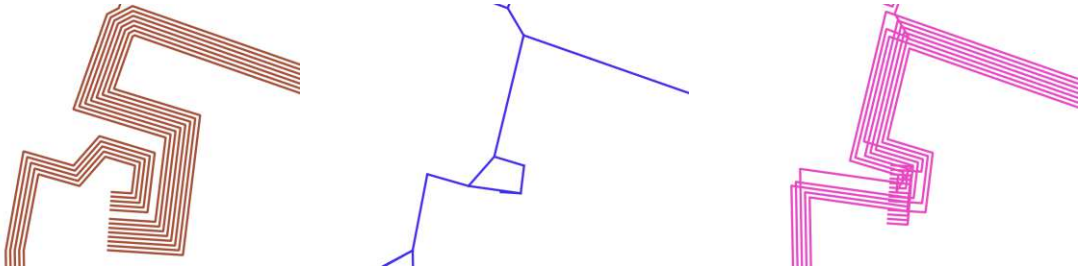
(a) Original (b) Edges Line (c) Generalized 1x

Figure 5.12: Test Case 8.

Another example is shown in Figure 5.13, where the algorithm fails entirely to construct a valid line graph. In this case, the original lines form a spiral-shaped outline. However, the algorithm misinterprets the geometry, and the resulting line graph incorrectly forms a closed circular structure. Due to this ambiguity in the line graph, the rendering phase fails at all scales.

Line Graph Construction – Semantic Issues

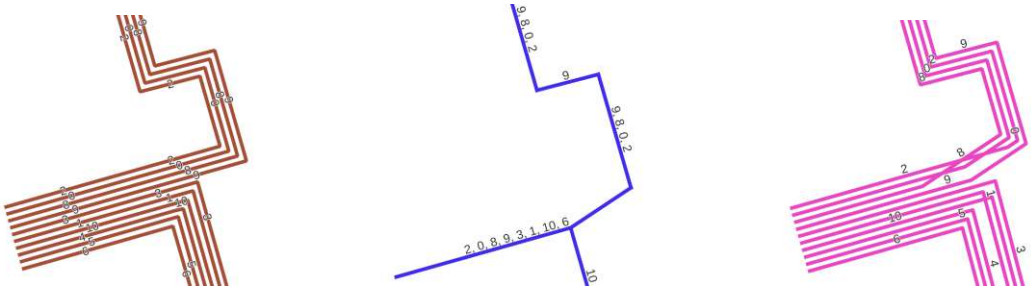
This class of problems also originates from the line graph construction, but unlike geometric issues, it stems from missing or ambiguous semantic information required to accurately reconstruct the original line order.



(a) Original (b) Edges Line (c) Generalized 1x

Figure 5.13: Test Case 10.

An example is shown in Figure 5.14, where the algorithm fails to restore the correct line sequence, resulting in additional crossings in the output (Figure 5.14c). As illustrated in Figure 5.14b, the line identifiers are incorrectly assigned. While the correct order for the first and second segments (from left to right) is `2, 0, 8, 9`, the subsequent segments are misordered as `9, 2, 0, 8` and then incorrectly flipped back to `2, 0, 8, 9`. This inconsistent ordering leads to a cluttered and misleading rendering.



(a) Original (b) Edges Line (c) Generalized 1x

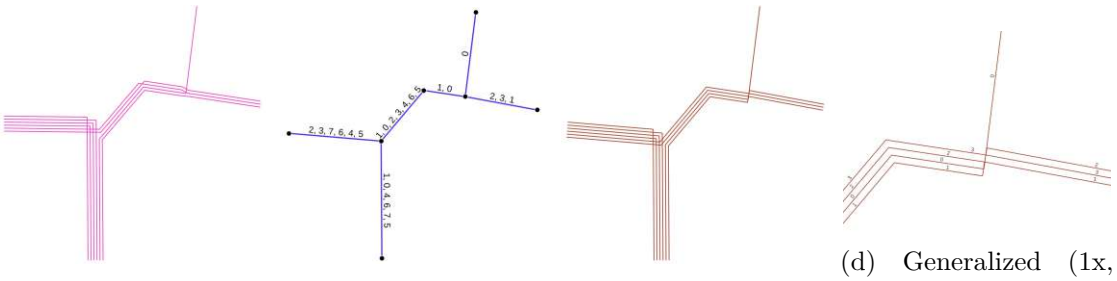
Figure 5.14: Test Case 43

In the next example, shown in Figure 5.15, the geometry is successfully reconstructed to match the original input. However, in the output rendering (Figure 5.15c), a shift becomes apparent on the right-hand side, specifically affecting the final segments.

Upon inspecting the line graph, we observe that the last segment is correctly tracked with the line order `2, 3, 1`. However, in the penultimate segment, some line identifiers are missing: only `1, 0` are detected. During the reconstruction phase, the line order is inherited from the preceding segment, where the lines appear as `1, 0, 2, 3`.

This leads to two key issues:

- The crossing between lines `2` and `3` is not detected by the algorithm and thus cannot be replicated in the output.
- The straight-line alignment fails to preserve the expected geometry. Specifically, the segments `3, 2, 0, 1` and `2, 3, 1` do not align as expected. Ideally, lines `2-3` and `3-2` would align to form a crossing, while lines `0-1` would remain parallel, resulting in minimal misalignment.



(a) Original input (b) Line graph (c) Generalized (1x) (d) Generalized (1x, zoomed)

Figure 5.15: Comparison of original image, line extraction, and simplification results.

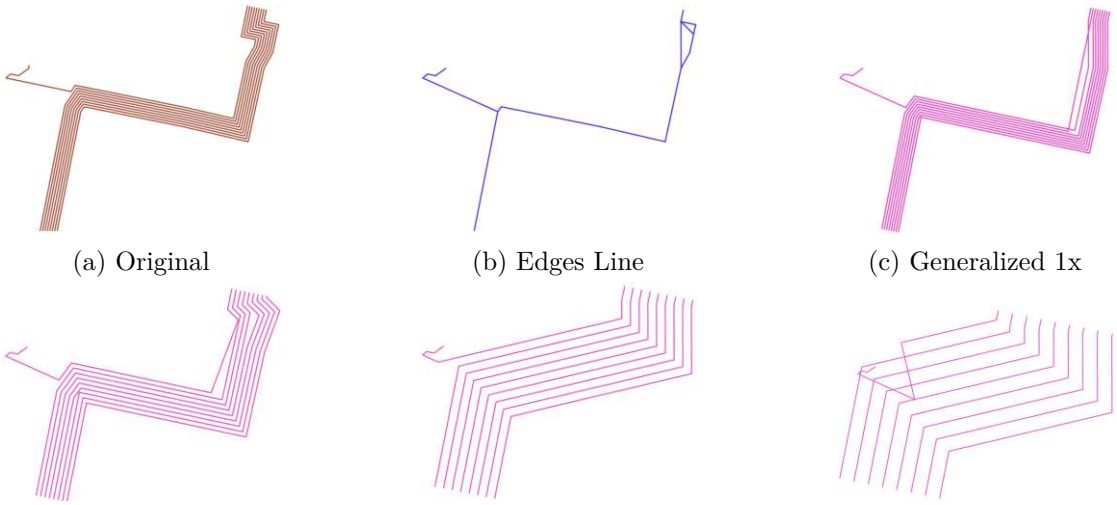
Render Issues

Another issue arises in the rendering phase, as illustrated in Figure 5.16. At scales **4x** and **8x**, the geometry deviates noticeably from the original form observed at **1x** and **2x**. Subjectively, one could argue that the reduced complexity results in a clearer visual representation. However, the simplification is arbitrary and not grounded in any consistent rule or constraint, which undermines the reliability of the output.

The root cause is the Shapely offset function, which fails to correctly render the L-shaped geometry in this case. As a fallback, our algorithm inserts a straight-line connection, which alters the original shape. The reasons behind the offset failure require further investigation, and a more robust alternative rendering approach must be developed.

Another issue is illustrated in Figure 5.17, which highlights the potential need for a dedicated line simplification step. At the **8x** scale, one line forms an acute angle. The U-turn present in the original geometry becomes increasingly compressed as the scale increases, particularly for the innermost lines. This results in an extremely narrow gap that cannot be reliably preserved at higher scales.

In this case, we observe that the bottom-most line appears simplified. However, this is not the result of an intentional simplification step. Rather, it is a byproduct of the cut-and-extend method used in the rendering process, which happens to eliminate the self-intersection in this particular instance.



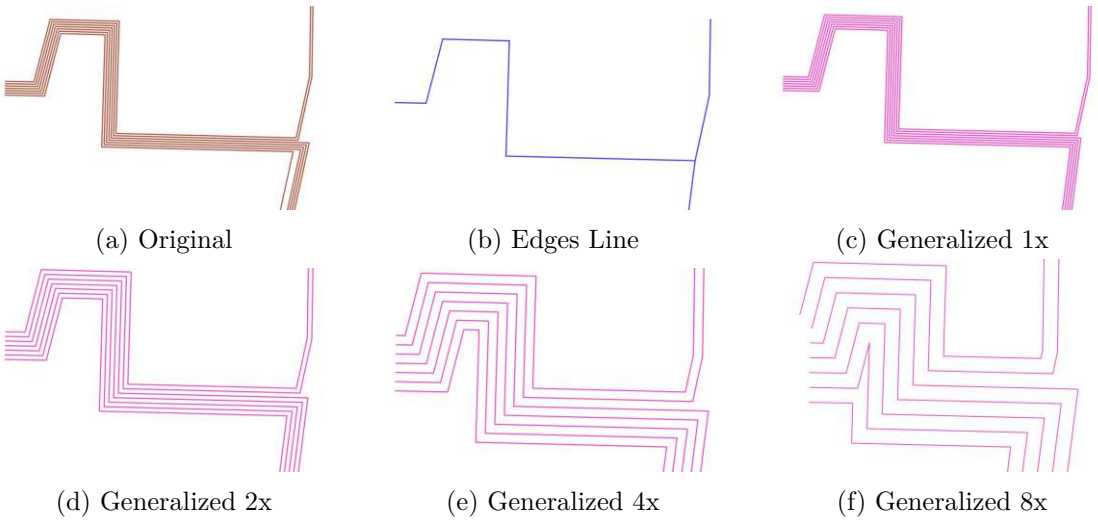
(a) Original (b) Edges Line (c) Generalized 1x

(d) Generalized 2x (e) Generalized 4x (f) Generalized 8x

Figure 5.16: Test Case 33.

This highlights the need to define clear criteria for when line simplification should be applied. Specifically, it must be determined whether features such as U-turns carry meaningful information for the reader or merely represent geometric artifacts.

If such features, as the U-turn, do not add relevant information to the drawing, they could be flattened into straight segments to simplify the structure, thereby enabling valid output even at higher scales.



(a) Original (b) Edges Line (c) Generalized 1x

(d) Generalized 2x (e) Generalized 4x (f) Generalized 8x

Figure 5.17: Test Case 38.

Route Issues

An additional problem arises during the path construction phase, which results in the extreme outlier values reported in the global analysis (Section 5.3) and substantially contributes to the high number of missing lines in the output.

As discussed in Subsection 4.5.4, paths are constructed using shortest paths between source and target points in the line graph. However, when the structure of the original path is not captured during the line graph construction, such a path cannot be reconstructed. In some cases, no path can be found at all, as the source and target points lie in separate, unconnected components.

An extreme example of this situation is illustrated in Figure 5.18, where the resulting Fréchet distance between the curves spans several hundred meters. The original path is shown in red, while the alternative route, which constitutes the output path, is shown in black. Examining the area around the original path reveals the root cause: the underlying line graph, depicted in blue in Figure 5.18, fails to reflect the correct connection. The nodes are not linked, making it impossible to reconstruct the original route.

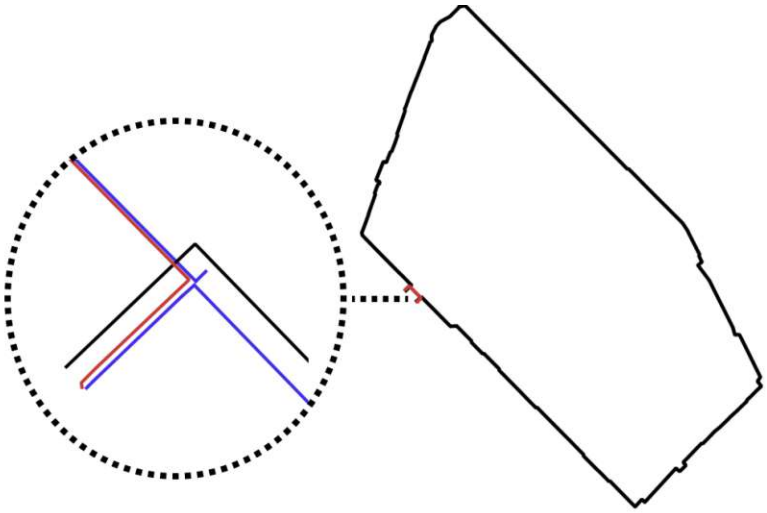


Figure 5.18: Example from a real-world dataset where an excessively long path is selected

5.4.5 Discussion of Local Test Cases

During the evaluation of the 51 local test cases, we identified several errors that emerged during the generalization process. Although we have shown that the algorithm can produce variable outputs at different scales, thus fulfilling requirement **R3**, additional problems arose at smaller scales, particularly at the 8x factor. 40 of the test cases passed at least the 1x, 2x, and 4x scales, but only 21 test cases passed at all four scales. A subjective assessment of the test cases revealed that even among these, most contained small to medium errors. Eight out of 51 test cases did not pass at any scale, which aligns

with the subjective assessment: they contain severe errors, and the resulting drawings often obscure the relationship to the original layout.

Based on this, we assess requirement **R2** as only partially fulfilled. The algorithm can produce acceptable output in some cases, but most drawings contain small to substantial errors. Many issues occur at the 8x scale, where rendering space becomes limited. Consequently, requirement **R6** is also only partially fulfilled. A strategy to displace features is needed to maintain readability at smaller scales.

Order preservation generally worked well, with only a few detected errors. Therefore, we consider requirement **R9** as rather fulfilled than only partially fulfilled. Some minor corrections could address the remaining issues.

As previously discussed, a more advanced displacement strategy is necessary. Requirements **R7** and **R11** are currently not met, as the algorithm does not adequately account for additional space requirements during the generalization process.

The requirement to prioritize readability over geographic accuracy (**R8**) is difficult to assess directly. During development, no trade-off situations arose in which geographic accuracy had to be sacrificed for readability. However, as illustrated in the example in Subsection 5.4.4, a simplification step could potentially improve the readability of the drawing. At present, the requirement does not specify how much simplification is acceptable or to what extent such steps can be applied without compromising the meaningfulness of the output for users. This highlights the need for further studies on usability or expert feedback to better define and constrain this requirement.

5.5 Runtime Performance

As the final step in the evaluation, we examine the algorithm's performance through a technical experiment. We measure the runtime of each processing phase and provide a more detailed analysis of the line graph construction phase, which is the most time-consuming. The experiment was conducted on a personal machine running Ubuntu 22.04 LTS (64-bit), equipped with an AMD Ryzen 5 5600X (6 cores @ 3.7–4.6 GHz) and 32 GB of RAM.

5.5.1 Overall Runtime Results

The algorithm processes the complete dataset, comprising approximately 433,000 input lines, in under 1 hour and 57 minutes. The most time-consuming phase is line graph generation, which takes approximately 1 hour and 9 minutes. The second most time-intensive phase is shortest path calculation, with a runtime of around 40 minutes. All remaining phases contribute only marginally to the total runtime. For derivation of the asymptotic runtimes see Section 4.6.

Phase	Measured time (s)	Asymptotic runtime (4.6)
Read input data	13.89 s	-
Merge lines components	53.88 s	$O(n \log c + c \log c)$
Presort Lines	0.58 s	$O(n \log n)$
Build line graph	4140.78 s	$O(nkP \log P \log n)$
Line orders	85.32 s	$O(EM)$
Graph cleanup	17.72 s	$O(N^2)$
Contracted degree 2 nodes	10.15 s	$O(V + E)$
Path Routing	2423.50 s	$O(n \cdot (V_2 + E_2) \cdot \log(V_2))$
Derived missing line orders	1.21 s	$O(nPM^2)$
Aligned straight line edges	8.02 s	$O((E_2 + V_2) \cdot PM^2)$
Offset lines	257.13 s	$O(nP)$
Total	7012.18 s	

Table 5.5: Total Runtime Results

5.5.2 Line Graph Construction Runtime

Since the line graph construction is the most expensive phase, we performed a detailed analysis on the iterative build step. We measure the metrics every 1,000 lines processed, starting from approximately 300,000 input lines. We expected the total comparison of the edge versus the inserted segment to be the main driver of the total runtime.

Our first observation was that the time required to process each 1,000-line batch decreased significantly as processing progressed. Since the lines are sorted by length, the longest lines are processed first. As a result, the early batches involve inserting more segments

and performing more comparisons. The plot in Figure 5.19 illustrates that the number of processed lines appears to grow exponentially, highlighting line length as a major runtime factor. On the right, we plotted the average number of comparisons per second. While there is a general upward trend, the curve collapses several times, most notably at the end, likely because as more lines are inserted, the time per insertion increases, while the number of comparisons per operation decreases.

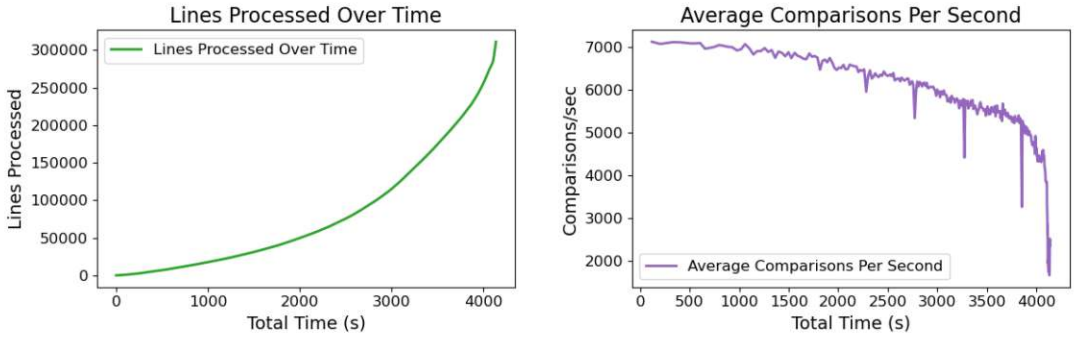


Figure 5.19: Runtime behavior over line insertions and average comparisons

In the next set of plots (Figure 5.20), we focus on the number of comparisons. The left plot shows a nearly linear relationship between the number of comparisons and total time, supporting the hypothesis that comparison operations are the main contributor to runtime. The right plot shows that as processing continues, the number of comparisons decreases, since the lines become shorter, producing a curve resembling a logarithmic-like function.

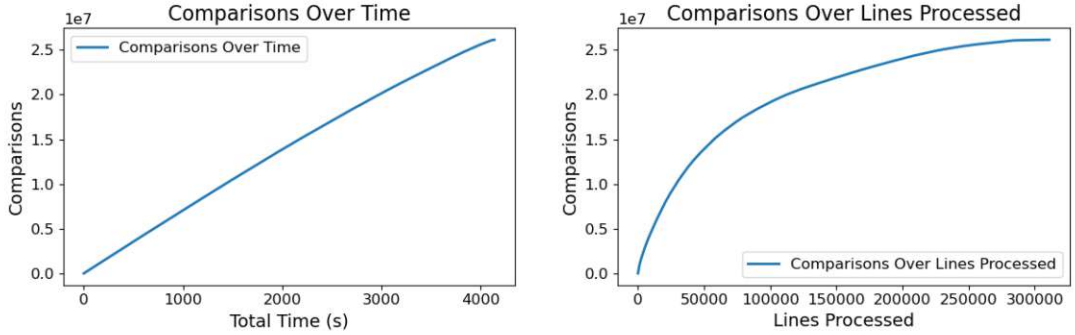


Figure 5.20: Total comparisons

Regarding memory usage, we observe a peak of approximately 3 GB at the end of the line graph phase. Memory usage increases gradually over time, and the number of lines processed per time unit decreases, correlating with trends seen in Figure 5.19 and Figure 5.20.

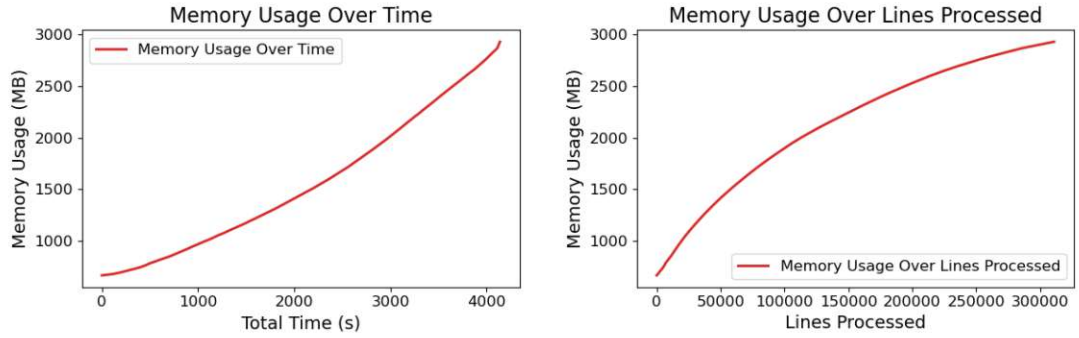


Figure 5.21: Memory usage during line graph construction

A similar pattern emerges when observing the size of the line graph. This confirms that the graph structure and its associated metadata are the primary consumers of memory. Notably, the nodes and edges grow nearly at a 1:1 ratio.

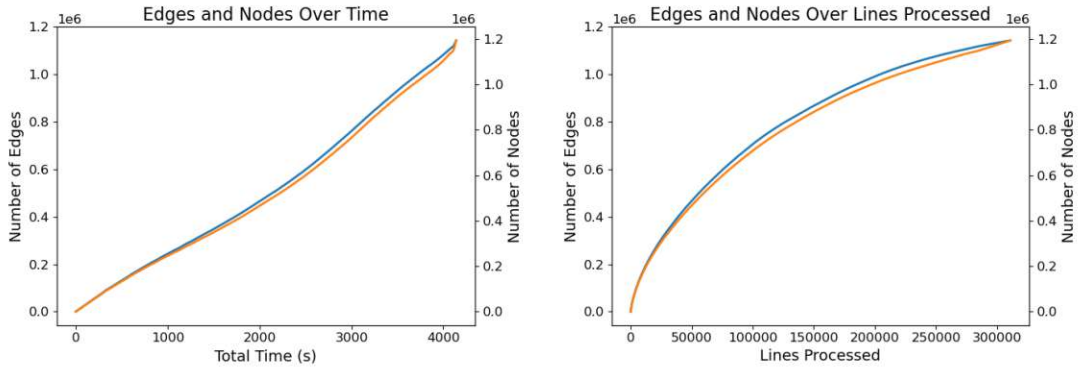


Figure 5.22: Total comparisons

5.5.3 Shortest Path Calculation Runtime

As discussed in Subsection 4.5.4 and Subsection 5.4.4, the current routing implementation encounters challenges when ambiguous paths are generated, particularly when the source and target nodes reside in different graph components. Additionally, as seen in the runtime results, such cases are computationally expensive, as the algorithm must explore an entire subgraph before terminating.

To address these issues, several strategies were proposed in Subsection 4.5.4:

- A straightforward approach would be to discard paths that are significantly longer than the original. While easy to implement, this does not reduce computation time, as the paths must still be computed.

- Another option would be to introduce a cutoff logic to terminate the calculation once a certain threshold is reached.

- A more advanced solution would incorporate a heuristic that guides the search based on Euclidean distance and metadata embedded in the line graph edges.

5.5.4 Discussion of Runtime Results

Given the upper bound for runtime of 48 hours, as defined by **R12**, the algorithm clearly meets this requirement with a total runtime of just under 2 hours. The implementation is based on pure Python and utilizes specialized libraries for key components such as spatial indexing (via R-tree), graph construction, and pathfinding. Therefore, requirement **R1** is also satisfied.

To further improve runtime, parallelization strategies, as outlined in Subsection 4.4.8 could be applied. The main performance bottleneck lies in the match calculation during line graph construction. This step involves approximately 26 million segment-to-segment comparisons. With a rate of about 4,500–7,000 comparisons, we reach the total runtime of approx. one hour. Performance could be improved by either reducing the number of comparisons through a better selection algorithm or by increasing the throughput of the comparison operations. We expect the latter to offer more potential, as 26 million is already a relatively small number. The individual comparison operations could be improved by replacing Python level by using compiled extensions (e.g., Cython), or delegating process-heavy operations to low-level calls, e.g., Rust or C++. A potential point of discussion is whether requirement **R1** can be relaxed to allow the integration of compiled code, which could lead to performance gains.

Another performance bottleneck is the path calculation, where a standard Dijkstra run is executed for each power line. This step is currently necessary to account for labeling errors introduced during the line graph construction phase. However, it results in unnecessarily long runtimes when the start and end points are not directly connected. A more efficient strategy needs to be developed to handle such labeling errors while avoiding unnecessary path searches.

5.6 Fullfilment of Requirements

To conclude the evaluation, we assess the fulfillment of the defined requirements, with the gained insights from the previous evaluation steps.

R1 – Implementation in Python (R1)

Fulfilled
This requirement is fulfilled, as the entire implementation is written in Python and uses Python libraries. For future performance improvements, it should be evaluated

whether this requirement can be relaxed, allowing certain performance-critical parts to be delegated to compiled languages.

R2 – Generalization for Readability (R2)

Partially Fulfilled
The local test case evaluation shows that, while the algorithm produces readable drawings in some instances, many outputs contain errors classified as displacement, geometric issues, semantic issues, rendering problems, or routing errors. In the global evaluation, less than half of the output lines pass all test metrics, highlighting the need for further improvement. Therefore, this requirement is considered partially fulfilled.

R3 – Configurable Output Scale (R3)

Fulfilled
This requirement is fulfilled: the output scale can be passed as a parameter and is properly respected by the algorithm. Although the evaluation showed that smaller scales tend to worsen output readability, the requirement itself is fully met. However, it may be beneficial to define a lower bound for this parameter to prevent configurations that lead to poor visual quality. This limitation should be further evaluated and potentially integrated into the system.

R4 – Accept Geodatabase as Input (R4)

Fulfilled
The algorithm accepts both geodatabases and shapefiles as input and therefore fulfills the requirement.

R5 – Support for Target Scale 1:2000 (R6)

Partially Fulfilled
Although the algorithm supports output intended for display at a scale of 1:2000, the evaluation has shown that this zoom level presents particular challenges. At this scale, the output often suffers from significantly increased clutter and limited available space, leading to overlapping elements and reduced readability. The need for a dedicated displacement strategy has become evident, as it would help resolve spatial conflicts and improve clarity, especially at lower detail levels. Therefore, this requirement is considered only partially fulfilled.

R6 – Fixed Size for Transformers (2×6m) (R7)

Not Fulfilled
This requirement was later extended by Requirement **R11**. Due to the dominant complexity of the line graph construction step, and because supporting fixed-size elements would require a dedicated displacement strategy, which is also a nontrivial problem, this

aspect was not addressed and is left for future work. Therefore, the requirement is not fulfilled.

R7 – Prioritization of Readability over Geographic Accuracy (R8)

Further Specification Needed
As discussed above, no clear trade-off situations arose where geographic accuracy had to be sacrificed for readability. However, the example in Subsection 5.4.4 suggests that simplification could improve readability, raising the question of how much deviation is acceptable. The requirement lacks a concrete definition, when the application of such simplification is desired.

R8 – Preservation of Line Ordering (R9)

Partially Fulfilled
Line order preservation generally works well, with only a few detected errors. While these issues are minor and could likely be corrected with small adjustments, they still prevent full compliance. Therefore, this requirement is considered partially fulfilled, though it is close to being fully met.

R9 – Support for Line and Point Data (R10)

Partially Fulfilled
The algorithm generally supports the generalization of line and point data. Although currently only the housing connections and components layers are considered during generalization. Therefore, the requirement is partially fulfilled.

R10 – Configurable Element Space Allocation (R11)

Not Fulfilled
This requirement extends **R7**. As stated before, a displacement strategy is needed to fulfill it, which was left out of scope and not implemented in this work.

R11 – Runtime Equal To or Faster Than Legacy (48h) (R12)

Fulfilled
Given the upper runtime limit of 48 hours defined in **R12**, the algorithm clearly fulfills this requirement, completing execution in just under 2 hours. Nevertheless, additional optimization strategies have been presented for potential future improvements.

R12 – Selective Feature Removal Support (R13)

Not Fulfilled
Support for selective feature removal is not implemented, as it was outside the scope of this work.

CHAPTER 6

Conclusion

This thesis presented the development and evaluation of an algorithm for generalizing complex real-world network data, following the Design Science Research (DSR) methodology. Wiener Netze served as the primary stakeholder, providing the real-world dataset and continuous feedback throughout the iterative development process.

The proposed generalization algorithm was implemented in Python, as required by **R1**, and structured into three main phases. The first phase involved loading data from a geodatabase **R4** and preprocessing, where the input line and point data were prepared and cleaned. A line graph was constructed in the second phase to detect parallel line segments and form the underlying power line network. This approach is closely related to the field of map construction and was primarily based on the work of Ahmed and Wenk [3]. The last phase rendered the final lines by splitting individual line segments from the line graph and offsetting them according to the desired line spacing, which fulfills requirement **R3**.

The algorithm was evaluated using three key metrics. The continuous Fréchet distance was employed to measure the geometric deviation between corresponding input and output lines. A crossing metric was used to detect newly introduced intersections, which should be minimized to preserve the structural integrity of the network. Finally, an acute angle metric was introduced to count line segments forming angles below 30 degrees, as such acute angles are considered undesirable artifacts in schematic network representations.

The evaluation revealed that a significant proportion of lines were not rendered, primarily due to a high number of unconnected components within the constructed line graph. Related routing issues were identified, where incorrect paths were selected, ultimately leading to inaccurate representations of parts of the original network. Therefore, requirements Preservation of Line Ordering (**R9** and Generalization for Readability **R2** are only partially met.

Moreover, the need for a global displacement and line simplification strategy was highlighted to enable generalization at higher scales. This is particularly important to support the target scale of 1:2000 better and to fulfill the requirements **R8**, **R6**, and **R11**.

In terms of performance, the algorithm successfully processed the entire dataset in 1 hour and 56 minutes, with a peak memory usage of approximately 3 GB. This result falls within the performance bounds defined by the requirement in item **R12**, demonstrating the practical feasibility of the approach for large-scale real-world datasets, in terms of total runtime.

Lastly, some requirements could not be fully met due to scope limitations, including Selective Feature Removal Support (**R13**) and Support for Line and Point Data (**R10**).

6.1 Future Work

In addition to the current evaluation methods, the skewness metric could be used. It measures the quality of the drawing by determining the minimum number of lines that must be removed to eliminate all crossings. This approach shifts the focus away from penalizing every intersecting line equally and instead highlights the specific lines that disproportionately contribute to visual clutter.

Regarding line graph construction, the continuity of the graph must be improved to enable more connections between paths, thereby reducing extreme outlier values and missing lines. Brosi and Bast [15] propose a technique where lines are inserted iteratively in multiple rounds until a threshold is reached. A similar approach could be adopted to enhance network reconstruction.

However, this would increase computational costs, as the line graph construction phase already represents the most time-consuming part of the algorithm. Performance optimizations, such as improving match calculations or employing Python compilers and accelerators like PyPy[12] or Numba[26], could improve the runtime. Alternatively, the matching component could be ported to a compiled language such as C++ or Rust.

Another issue lies in line detection: the current method considers all (semi-)parallel lines within a 5-meter radius, which can lead to ambiguous connections. A refined approach would limit merging to line bundles where every individual line remains within the intended original line distance of about 0.2m. This modification would prevent unintended groupings while still accommodating wide bundles, as the largest observed bundle contained up to 23 parallel lines.

For line order preservation, it should be evaluated whether the constraint from item **R9** can be relaxed. Fixating line orders only at source and sink nodes could allow intermediate orders to be optimized for improved readability. Alternatively, more sophisticated methods for detecting line orders could be developed if strict preservation remains necessary.

For the routing step, a more connected line graph would naturally lead to better routing results. Nevertheless, enhancements such as implementing cutoff thresholds and heuristic-

guided shortest path calculations should be considered. Additionally, research into fallback strategies for connecting previously unconnected segments would be valuable to mitigate errors originating from graph construction limitations.

The integration of a global displacement strategy is essential to support higher levels of generalization. Such a strategy would dynamically adjust line positions based on spatial proximity while preserving both topological correctness and geometric structure. Furthermore, the requirement item **R11** could be fully addressed by integrating point elements into the displacement phase. A promising starting point for developing such a displacement strategy could be the concept of Smooth Orthogonal Layouts, as introduced by Bekos, Kaufmann, Kobourov, and Symvonis [11].

Finally, requirement **R8**, the prioritization of readability over geographic accuracy, should be further investigated and specified to provide clear guidelines for line simplification. Methods such as the modified Ramer-Douglas-Peucker algorithm[18] described in [25] could be applied to simplify degree-2 paths while preserving overall structure. Strategies for removing self-intersections must also be researched. Alternatives to the current shapely line offset algorithm should be considered if better performance or fewer artifacts can be achieved. In addition, methods for network schematization [9, 8] may offer valuable techniques for further enhancing visual quality. Notably, Sadahiro, Tanabe, Pierre, and Fujii[33] propose that acute bends can be indirectly resolved by discretizing line directions, suggesting that incorporating schematization techniques could also contribute to improving network structure and reducing geometric artifacts.

Overview of Generative AI Tools Used

Grammarly

Grammarly was used throughout the thesis as a browser plugin to correct spelling and grammar. Suggestions were applied when appropriate to improve the writing style.

ChatGPT

ChatGPT was also used to review and correct the text. Prompts took the form: "Correct grammar and spelling. Improve clarity, without changing the original meaning or tone [text]".

ChatGPT was used for translating the acknowledgements from German to English with the prompt "Translate this acknowledgements section of a thesis from English to German [text]".

List of Figures

List of Tables

List of Algorithms

Bibliography

[1] Mahmuda Ahmed, Sophia Karagiorgou, Dieter Pfoser, and Carola Wenk. "A comparison and evaluation of map construction algorithms using vehicle tracking data". In: *Geoinformatica* 19 (2015), pp. 601–632.

[2] Mahmuda Ahmed, Sophia Karagiorgou, Dieter Pfoser, and Carola Wenk. *Map Construction Algorithms*. Springer, 2015. ISBN: 978-3-319-25164-6. DOI: 10.1007/978-3-319-25166-0. URL: https://doi.org/10.1007/978-3-319-25166-0.

[3] Mahmuda Ahmed and Carola Wenk. "Constructing Street Networks from GPS Trajectories". In: *Algorithms - ESA 2012 - 20th Annual European Symposium, Ljubljana, Slovenia, September 10-12, 2012. Proceedings*. Ed. by Leah Epstein and Paolo Ferragina. Vol. 7501. Lecture Notes in Computer Science. Springer, 2012, pp. 60–71. DOI: 10.1007/978-3-642-33090-2_7.

[4] Helmut Alt, Alon Efrat, Günter Rote, and Carola Wenk. "Matching planar maps". In: *Journal of algorithms* 49.2 (2003), pp. 262–283.

[5] Helmut Alt and Michael Godau. "Computing the Fréchet distance between two polygonal curves". In: *Int. J. Comput. Geom. Appl.* 5 (1995), pp. 75–91. DOI: 10.1142/S0218195995000064.

[6] Helmut Alt and Michael Godau. "Measuring the Resemblance of Polygonal Curves". In: *Proceedings of the Eighth Annual Symposium on Computational Geometry, Berlin, Germany, June 10-12, 1992*. Ed. by David Avis. ACM, 1992, pp. 102–109. DOI: 10.1145/142675.142699.

[7] Hannah Bast, Patrick Brosi, and Sabine Storandt. "Efficient generation of geographically accurate transit maps". In: *Acm transactions on spatial algorithms and systems (tsas)* 5.4 (2019), pp. 1–36.

[8] Hannah Bast, Patrick Brosi, and Sabine Storandt. "Metro Maps on Flexible Base Grids". In: *Proceedings of the 17th International Symposium on Spatial and Temporal Databases, SSTD 2021, Virtual Event, USA, August 23-25, 2021*. Ed. by Erik Hoel, Dev Oliver, Raymond Chi-Wing Wong, and Ahmed Eldawy. ACM, 2021, pp. 12–22. DOI: 10.1145/3469830.3470899.

[9] Hannah Bast, Patrick Brosi, and Sabine Storandt. “Metro Maps on Octilinear Grid Graphs”. In: *Computer Graphics Forum*. Vol. 39. 3. 2020, pp. 357–367. DOI: 10.1111/CGF.13986.

[10] Norbert Beckmann, Hans-Peter Kriegel, Ralf Schneider, and Bernhard Seeger. “The R*-Tree: An Efficient and Robust Access Method for Points and Rectangles”. In: *Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data*. SIGMOD ’90. Atlantic City, New Jersey, USA: Association for Computing Machinery, 1990, pp. 322–331. ISBN: 0897913655. DOI: 10.1145/93597.98741.

[11] Michael A. Bekos, Michael Kaufmann, Stephen G. Kobourov, and Antonios Symvonis. “Smooth Orthogonal Layouts”. In: *Graph Drawing - 20th International Symposium, GD 2012, Redmond, WA, USA, September 19-21, 2012, Revised Selected Papers*. Ed. by Walter Didimo and Maurizio Patrignani. Vol. 7704. Lecture Notes in Computer Science. Springer, 2012, pp. 150–161. DOI: 10.1007/978-3-642-36763-2_14.

[12] Carl Friedrich Bolz, Antonio Cuni, Maciej Fijalkowski, and Armin Rigo. “Tracing the meta-level: PyPy’s tracing JIT compiler”. In: *Proceedings of the 4th workshop on the Implementation, Compilation, Optimization of Object-Oriented Languages and Programming Systems, ICOOOLPS 2009, Genova, Italy, July 6, 2009*. Ed. by Ian Rogers. ACM, 2009, pp. 18–25. DOI: 10.1145/1565824.1565827.

[13] Jan vom Brocke, Alan Hevner, and Alexander Maedche. “Introduction to Design Science Research”. In: *Design Science Research. Cases*. Ed. by Jan vom Brocke, Alan Hevner, and Alexander Maedche. Cham: Springer International Publishing, 2020, pp. 1–13. ISBN: 978-3-030-46781-4. DOI: 10.1007/978-3-030-46781-4_1.

[14] Mattia Broilo, Nicola Piotto, Giulia Boato, Nicola Conci, and Francesco G. B. De Natale. “Object Trajectory Analysis in Video Indexing and Retrieval Applications”. In: *Video Search and Mining*. Ed. by Dan Schonfeld, Caifeng Shan, Dacheng Tao, and Liang Wang. Vol. 287. Studies in Computational Intelligence. Springer, 2010, pp. 3–32. DOI: 10.1007/978-3-642-12900-1_1. URL: https://doi.org/10.1007/978-3-642-12900-1%5C_1.

[15] Patrick Brosi and Hannah Bast. “Large-scale generation of transit maps from OpenStreetMap data”. In: *The cartographic journal* 60.4 (2023), pp. 342–366.

[16] Kevin Buchin, Maike Buchin, and Yusu Wang. “Exact algorithms for partial curve matching via the Fréchet distance”. In: *Proceedings of the Twentieth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2009, New York, NY, USA, January 4-6, 2009*. Ed. by Claire Mathieu. SIAM, 2009, pp. 645–654. DOI: 10.1137/1.9781611973068.71. URL: https://doi.org/10.1137/1.9781611973068.71.

[17] Giuseppe Di Battista, Peter Eades, Roberto Tamassia, and Ioannis G Tollis. *Graph drawing : algorithms for the visualization of graphs*. eng. An Alan R. Apt book. Upper Saddle River, NJ [u.a.]: Prentice Hall, 1999. ISBN: 0133016153.

[18] David H Douglas and Thomas K Peucker. “Algorithms for the reduction of the number of points required to represent a digitized line or its caricature”. In: *Cartographica* 10.2 (1973), pp. 112–122. DOI: 10.3138/FM57-6770-U75U-7727.

[19] Stefan Edelkamp and Stefan Schrödl. “Route Planning and Map Inference with Global Positioning Traces”. In: *Computer Science in Perspective: Essays Dedicated to Thomas Ottmann*. Springer, 2003, pp. 128–151.

[20] Thomas Eiter and Heikki Mannila. *Computing Discrete Fréchet Distance*. Tech. rep. CD-TR 94/64. Vienna, Austria: Christian Doppler Laboratory for Expert Systems, TU Vienna, 1994. URL: https://www.kr.tuwien.ac.at/staff/eiter/et-archive/files/cdtr9464.pdf.

[21] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise”. In: *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96), Portland, Oregon, USA*. Ed. by Evangelos Simoudis, Jiawei Han, and Usama M. Fayyad. AAAI Press, 1996, pp. 226–231. URL: http://www.aaai.org/Library/KDD/1996/kdd96-037.php.

[22] Rudolf Fleischer and Colin Hirsch. “Graph Drawing and Its Applications”. In: *Drawing Graphs: Methods and Models*. Ed. by Michael Kaufmann and Dorothea Wagner. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 1–22. ISBN: 978-3-540-44969-0. DOI: 10.1007/3-540-44969-8_1.

[23] Antonin Guttman. “R-Trees: A Dynamic Index Structure for Spatial Searching”. In: *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data*. SIGMOD ’84. Boston, Massachusetts: Association for Computing Machinery, 1984, pp. 47–57. ISBN: 0897911288. DOI: 10.1145/602259.602266.

[24] Kamran Khan, Saif ur Rehman, Kamran Aziz, Simon Fong, Sababady Sarasvady, and Amrita Vishwa. “DBSCAN: Past, present and future”. In: *The Fifth International Conference on the Applications of Digital Information and Web Technologies, ICADIWT 2014, Chennai, India, February 17-19, 2014*. IEEE, 2014, pp. 232–238. DOI: 10.1109/ICADIWT.2014.6814687.

[25] Johannes Kopf, Maneesh Agrawala, David Bargeron, David Salesin, and Michael Cohen. “Automatic generation of destination maps”. In: *Acm transactions on graphics (tog)* 29.6 (2010), pp. 1–12.

[26] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. “Numba: a LLVM-based Python JIT compiler”. In: *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC, LLVM 2015, Austin, Texas, USA, November 15, 2015*. Ed. by Hal Finkel. ACM, 2015, 7:1–7:6. DOI: 10.1145/2833157.2833162.

[27] Sogo Mizutani, Y Kim, D Yamamoto, and Naohisa Takahashi. “Automatic generation method for geographically accurate bus route maps from bus stops”. In: *Proceedings of the geoprocessing* (2022), pp. 19–24.

[28] Ken Peffers, Tuure Tuunanen, Marcus A Rothenberger, and Samir Chatterjee. "A design science research methodology for information systems research". In: *Journal of Management Information Systems* 24.3 (2007), pp. 45–77.

[29] O. Pueyo, X. Pueyo, and G. Patow. "An overview of generalization techniques for street networks". In: *Graphical models* 106 (2019), p. 101049. ISSN: 1524-0703. DOI: 10.1016/j.gmod.2019.101049.

[30] HELEN C. Purchase. "Metrics for Graph Drawing Aesthetics". In: *Journal of visual languages computing* 13.5 (2002), pp. 501–516. ISSN: 1045-926X. DOI: 10.1006/jvlc.2002.0232.

[31] Urs Ramer. "An iterative procedure for the polygonal approximation of plane curves". In: *Comput. Graph. Image Process.* 1.3 (1972), pp. 244–256. DOI: 10.1016/S0146-664X(72)80017-0. URL: https://doi.org/10.1016/S0146-664X(72)80017-0.

[32] Anne Ruas. "Map Generalization". In: *Encyclopedia of GIS*. Ed. by Shashi Shekhar and Hui Xiong. Boston, MA: Springer US, 2008, pp. 631–632. ISBN: 978-0-387-35973-1. DOI: 10.1007/978-0-387-35973-1_743.

[33] Yukio Sadahiro, Takahito Tanabe, Maxime Pierre, and Koichi Fujii. "Computer-aided design of bus route maps". In: *Cartography and geographic information science* 43.4 (2016), pp. 361–376. DOI: 10.1080/15230406.2015.1077162.

[34] Jochen Schiewe. "Kartographische Generalisierung". In: *Kartographie: Visualisierung georäumlicher Daten*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2022, pp. 329–350. ISBN: 978-3-662-65441-5. DOI: 10.1007/978-3-662-65441-5_18.

[35] Erich Schubert, Jörg Sander, Martin Ester, Hans-Peter Kriegel, and Xiaowei Xu. "DBSCAN Revisited, Revisited: Why and How You Should (Still) Use DBSCAN". In: *ACM Trans. Database Syst.* 42.3 (2017), 19:1–19:21. DOI: 10.1145/3068335.

[36] Jantien Stoter, B Baella, CA Blok, D Burghardt, C Duchêne, M Pla, N Regnauld, and G Touya. *State-of-the-art of automated generalisation in commercial software*. EuroSDR Dublin, 2010.

[37] Guillaume Touya, Xiang Zhang, and Imran Lokhat. "Is deep learning the new agent for map generalization?" In: *International journal of cartography* 5.2–3 (2019), pp. 142–157. DOI: 10.1080/23729333.2019.1613071.

[38] Matthew Treinish, Ivan Carvalho, Georgios Tsilimigkounakis, and Nahum Sá. "rustworkx: A High-Performance Graph Library for Python". In: *Journal of open source software* 7.79 (2022), p. 3968. DOI: 10.21105/joss.03968.

[39] Colin Ware, Helen Purchase, Linda Colpoys, and Matthew McGill. "Cognitive Measurements of Graph Aesthetics". In: *Information visualization* 1.2 (2002), pp. 103–110. DOI: 10.1057/palgrave.ivs.9500013.

[40] Alexander Wolff. "Handbook of graph drawing and visualization". In: CRC press, 2013. Chap. 23 - Graph Drawing and Cartography, pp. 697–736.